

Matchings of the n -prism with one spoke or one rim
edge deleted:
two bridge theorems to recurrence-defined sequences
(Lean 4 machine-checked claim of record)

Aurora0134*

October 1, 2026

Abstract

Let $\text{aspoke}(n)$ and $\text{arim}(n)$ be the integer sequences with initial values 25, 86, 271, 876 and 26, 86, 274, 883 respectively, both satisfying the fourth-order recurrence $a(n+4) = 2a(n+3) + 4a(n+2) - a(n)$ of the prism graph's own matching sequence A102080 (same recurrence signature, different initial values). For every $n \geq 3$, the number of matchings of the n -prism graph $C_n \square P_2$ with one spoke (rung) edge deleted equals $\text{aspoke}(n-3)$, and with one rim (perimeter) edge deleted equals $\text{arim}(n-3)$; matchings are counted in the Hosoya sense, the empty matching included. Both theorems are formalised in Lean 4 (mathlib v4.34.0) and verified by the kernel with no `sorry`; `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. The pipeline's exclusivity review found no occupying record (reachable channels): the two sequences are unregistered and the graph-side bridge is graded first formalisation (Lean three-layer zero same-form); the value tier is new sequence, never new mathematics and never a new recurrence. This note is a priority claim of record, not a full paper.

1 Introduction

This note records two formalised bridge theorems produced by the AI4Math automated pipeline, in which LLM workers generate statements and proof scripts and the Lean 4 kernel is the sole adjudicator at every gate: the statement was frozen by hash before proving, and a separate audit re-computed

*Contact: via the GitHub account Aurora0134.

the symmetric difference, scanned the whole file for `sorry/admit` including comments, and re-printed the axiom dependencies [1]. The object is graph-side and combinatorial: the matching count (Hosoya index, edge-subset formulation, empty matching included) of the n -prism with a single edge deleted, for the two edge orbits of the prism (one spoke, one rim). The proof is a multi-stage assembly (split identity, structural decomposition, transfer/cell lemmas, strong induction on the sequences) built on four library modules that are themselves kernel-checked and sorry-free. The note is a priority claim of record, deposited for timestamping; the corresponding narrative paper is prepared separately and will reference this deposit. Verification strength is stated in Section 4. No literature survey is included—that expectation belongs to arXiv moderation and does not apply to a Zenodo deposit.

2 Statement

We fix notation. The prism graph on n vertices per layer is formalised as `prism n = cycleGraph n □ pathGraph 2` on the vertex type `Fin n × Fin 2`. The two defect slices delete one edge of column 0 with both endpoints kept: the spoke (rung) $((0,0), (0,1))$ gives `prismDelSpoke n`, and the rim (perimeter) edge $((0,0), (1,0))$ gives `prismDelRim n`. `hosoya G` counts the matchings of G as the number of edge subsets whose elements pairwise have disjoint endpoint sets, the empty matching included (Hosoya’s definition). The two sequences are defined by the same fourth-order recurrence with the stated initial values; index 0 corresponds to graph-side $n = 3$. The two theorems formalised and proved here are:

Theorem 1 (spoke-side bridge). *For every $n \in \mathbb{N}$ with $3 \leq n$,*

$$(\text{hosoya}(\text{prismDelSpoke } n) : \mathbb{Z}) = \text{aspoke}(n - 3).$$

Theorem 2 (rim-side bridge). *For every $n \in \mathbb{N}$ with $3 \leq n$,*

$$(\text{hosoya}(\text{prismDelRim } n) : \mathbb{Z}) = \text{arim}(n - 3).$$

The subscript $n - 3$ is natural-number truncated subtraction, exact on the hypothesis domain $3 \leq n$; the frozen statement transcribes the frozen natural-language wording verbatim and deliberately keeps this form.

Listing 1: Frozen statement snapshot (Lean 4, verbatim lines 36–98 of `formalized/statement.lean`, sha256 `d186ff16`), as an ordered subsequence with exactly three lines excluded, declared immediately below.

```

1  -- ===== 基座代码（逐字，禁改） =====
2
3  instance (n : ℕ) : DecidableRel (pathGraph n).Adj := fun _ _ =>
4    decidable_of_iff _ pathGraph_adj.symm
5
6  instance {α β : Type*} [DecidableEq α] [DecidableEq β] (G :
7    SimpleGraph α) (H : SimpleGraph β)
8    [DecidableRel G.Adj] [DecidableRel H.Adj] : DecidableRel (G
9    □ H).Adj := fun _ _ =>
10   decidable_of_iff _ boxProd_adj.symm
11
12 def edgeSupp {V : Type*} [DecidableEq V] [Fintype V] (e : Sym2 V)
13   : Finset V :=
14   (univ : Finset V).filter fun v => v ∈ e
15
16 def hosoya (G : SimpleGraph V) [Fintype V] [DecidableEq V] [
17   DecidableRel G.Adj] : ℕ :=
18   ((G.edgeFinset.powerset).filter fun s =>
19     (∀ e ∈ s, ∀ f ∈ s, e ≠ f → Disjoint (edgeSupp e) (edgeSupp f)
20     )).card
21
22 def prism (n : ℕ) : SimpleGraph (Fin n × Fin 2) := cycleGraph n
23   □ pathGraph 2
24
25 instance (n : ℕ) : DecidableRel (prism n).Adj :=
26   show DecidableRel (cycleGraph n □ pathGraph 2).Adj from
27   inferInstance
28
29 def prismDelSpoke (n : ℕ) (hn : 3 ≤ n) : SimpleGraph (Fin n × Fin
30   2) :=
31   (prism n).deleteEdges {Sym2.mk ((⟨0, by omega⟩, (0 : Fin 2))) ((⟨
32     0, by omega⟩, (1 : Fin 2)))}
33
34 instance (n : ℕ) (hn : 3 ≤ n) : DecidableRel (prismDelSpoke n hn)
35   .Adj :=
36   show DecidableRel ((prism n).deleteEdges
37     {Sym2.mk ((⟨0, by omega⟩, (0 : Fin 2))) ((⟨0, by omega⟩, (1 :
38       Fin 2)))}) .Adj from inferInstance
39
40 def prismDelRim (n : ℕ) (hn : 3 ≤ n) : SimpleGraph (Fin n × Fin
41   2) :=
42   (prism n).deleteEdges {Sym2.mk ((⟨0, by omega⟩, ⟨0, by omega⟩))
43     ((⟨1, by omega⟩, ⟨0, by omega⟩))}
44
45 instance (n : ℕ) (hn : 3 ≤ n) : DecidableRel (prismDelRim n hn).

```

```

Adj :=
33 show DecidableRel ((prism n).deleteEdges
34   {Sym2.mk ((⟨0, by omega⟩, ⟨0, by omega⟩)) ((⟨1, by omega⟩, ⟨0,
   by omega⟩)))}.Adj from inferInstance
35
36 def aspoke : ℕ → ℤ
37   | 0 => 25
38   | 1 => 86
39   | 2 => 271
40   | 3 => 876
41   | n + 4 => 2 * aspoke (n + 3) + 4 * aspoke (n + 2) - aspoke n
42
43 def arim : ℕ → ℤ
44   | 0 => 26
45   | 1 => 86
46   | 2 => 274
47   | 3 => 883
48   | n + 4 => 2 * arim (n + 3) + 4 * arim (n + 2) - arim n
49
50
51 /-- **主定理一 (spoke 侧桥定理) **: 对一切 `n ≥ 3`, 删一条 spoke 的
   棱柱的匹配数等于序列
52 `aspoke (n - 3)`. 其中 `aspoke` 为已冻结序列 (初值 25, 86, 271, 876
   , 四阶递推
53 `a(n+4) = 2·a(n+3) + 4·a(n+2) - a(n)`, 下标 0 对应图侧 n = 3)。 -/
54 theorem hosoya_prism_del_spoke : ∀ (n : ℕ) (hn : 3 ≤ n),
55   (hosoya (prismDelSpoke n hn) : ℤ) = aspoke (n - 3) := by
56
57 /-- **主定理二 (rim 侧桥定理) **: 对一切 `n ≥ 3`, 删一条 rim 边的棱
   柱的匹配数等于序列
58 `arim (n - 3)`. 其中 `arim` 为已冻结序列 (初值 26, 86, 274, 883, 同
   四阶递推)。 -/
59 theorem hosoya_prism_del_rim : ∀ (n : ℕ) (hn : 3 ≤ n),
60   (hosoya (prismDelRim n hn) : ℤ) = arim (n - 3) := by

```

This is lines 36–98 of the frozen statement file with exactly three lines excluded, all declared here: the two proof-body placeholder lines (line 92 and line 98, each the single line **sorry**, the formalisation department’s placeholders for the two bridge theorems, which this note’s Section 3 replaces with the real kernel-checked proofs), and the one-line separator comment at line 85, which is excluded only because the paper-lint listing gate rejects the token it contains inside listings and the line is otherwise a vacuous section marker. Everything else—the header of the appended block, both doc-comments, all base definitions, both instances and both theorem statements—is verba-

tim and in order. `inspect-tex.py`, the package-local checker copied under `audit/`, checks this mechanically: every remaining line occurs in the source in order, and the only lines missing from the block are those three. The real, sorry-free proofs follow in full.

3 Proof

The proof file imports four kernel-checked library modules (the split identity of the pipeline’s Phase 0, the Phase 1 structural lemmas, the ladder-side and prism-side recurrences) and assembles the two bridges in three lanes. Lane A (split): a general edge-deletion identity cuts the full prism count along the deleted edge into the two defect factors plus an endpoint-deleted factor. Lane B (structure): column-wise decompositions express the two defect counts through the intact-prism sequence and an auxiliary sequence, packaged as cell lemmas closed by `decide` over an eight-step library recurrence chain. Lane C (induction): the strong-induction closers `aspoke_strong/arim_strong` derive the sequence values from the recurrence, and the two frozen theorem heads are discharged from the $\mathbb{Z}$ -cast bridges by `omega`. The complete file (1857 lines) follows verbatim; it also ships in this deposit under `proofs/`.

Listing 2: Complete proof file of Theorems 1 and 2, verbatim lines 1–1857 (whole file) of `attempts/phase2/r3/bridge.lean` (sha256 399ee81f). Mixed line endings on the working disk (the file is CRLF throughout) are spliced LF-normalised per the chorded/ladder/hosoya precedent; the byte-exact original ships in `proofs/`.

```

1  import Mathlib
2  import Library.HosoyaPrismSplit
3  import Library.GraphsidePhase1
4  import Library.LadderHosoya
5  import Library.PrismHosoya
6
7  /-!
8  # Phase 2 r3 · 双桥定理 + 棱柱 R4 收口 (bridge.lean, 2026-10-01)
9
10 provenance:
11 - 任务契约 `attempts/phase2/r3/prompt.md` (Phase 2 第 4 采样位, 最后一
    个)。
12 - 基座 def (aspoke/arim/prismDelRim + instance) 逐字拷自冻结快照
13   `formalized/statement.lean` L64-L83 (宪条 2: 只许拷贝不许改)。
14 - 数学骨架 (契约 T4a-T4f) :  $p(n) = a(n)+2c(n)+a(n-2)$  四格分解;
15    $d_{rim}(n) = a(n)+c(n)$  (列旋转共轭);  $d_{spoke}(n) = p(n)-a(n-1)$  (Phase 0 分
    割

```

```

16 + 同构)；归纳桥走强归纳 + aspoke/arim 自身  $R_4(\mathbb{Z})$ 。库件 GO 全部经
    kernel
17 预核验（本文件编译即复验）。
18 - 战术承 r1/r2 模板：rfl 桥 (Fin.val_mk / pair 投影)、omega 断原子先放桥、
19 membership 经 `eset_mem_mk`/`lad_adj_iff`/`disjoint_esupp_iff` 线性化。
20 - n 一律写成 m+3 (Fin (m+3) 上字面量 1 与算术实例可用；Fin n 一般形无
21 OfNat 实例，r1/r2 同款移位口径)。
22 -/
23
24 open SimpleGraph Finset
25
26 -- ===== 基座 def (逐字拷自冻结快照 formalized/statement.lean, 勿改) =====
27
28 def prismDelRim (n : ℕ) (hn : 3 ≤ n) : SimpleGraph (Fin n × Fin 2) :=
29   (prism n).deleteEdges {Sym2.mk ((⟨0, by omega⟩), ⟨0, by omega⟩)) ((⟨1, by
    omega⟩), ⟨0, by omega⟩))}
30
31 instance (n : ℕ) (hn : 3 ≤ n) : DecidableRel (prismDelRim n hn).Adj :=
32   show DecidableRel ((prism n).deleteEdges
33     {Sym2.mk ((⟨0, by omega⟩), ⟨0, by omega⟩)) ((⟨1, by omega⟩), ⟨0, by omega
    34     35     36     37     38     39     40     41
    42     43     44     45     46     47     48
    49 namespace PrismR3
    50
    51 /-! ## $0 小工具 (Fin 2 行算术 / Fin (m+3) 模加一 / wrap 边) -/
    52
    53 -- Fin 2 行算术 (全枚举 decide)
    54 theorem one_sub_val : ∀ r : Fin 2, ((1 - r : Fin 2) : ℕ) = 1 - (r : ℕ)
    := by decide
    55 theorem one_sub_ne : ∀ r s : Fin 2, r ≠ s → (1 - r : Fin 2) ≠ 1 - s :=
    by decide
    56 theorem one_sub_eq_one : ∀ r : Fin 2, ((1 - r : Fin 2) : ℕ) = 1 ↔ r = 0
    := by decide
    57 theorem one_sub_eq_zero : ∀ r : Fin 2, ((1 - r : Fin 2) : ℕ) = 0 ↔ r = 1
    := by decide

```

```

58
59 -- Fin (m+3) 加一 (含环绕) 的 val 字符化
60 theorem fin_add_one_val (m : ℕ) (u v : Fin (m+3)) :
61   v = u + 1 ↔ (v : ℕ) = (u : ℕ) + 1 ∨ ((u : ℕ) = m + 2 ∧ (v : ℕ) = 0) :=
62   by
63     constructor
64     · intro h
65       have hc := congrArg Fin.val h
66       rw [Fin.val_add, Fin.val_one] at hc
67       rcases Nat.lt_or_ge ((u : ℕ) + 1) (m + 3) with hlt | hge
68       · rw [Nat.mod_eq_of_lt hlt] at hc
69         exact Or.inl hc
70       · have hu : (u : ℕ) = m + 2 := by omega
71         have hmod : ((u : ℕ) + 1) % (m + 3) = 0 := by
72           rw [show (u : ℕ) + 1 = m + 3 from by omega]
73           exact Nat.mod_self _
74         rw [hmod] at hc
75         exact Or.inr ⟨hu, hc⟩
76     · rintro (h | ⟨hu, hv⟩)
77       · apply Fin.val_injective
78         rw [Fin.val_add, Fin.val_one]
79         rw [show ((u : ℕ) + 1) % (m + 3) = (u : ℕ) + 1 from Nat.mod_eq_of_lt (
80           by omega)]
81         exact h
82       · apply Fin.val_injective
83         rw [Fin.val_add, Fin.val_one]
84         rw [show ((u : ℕ) + 1) % (m + 3) = 0 from by
85           rw [show (u : ℕ) + 1 = m + 3 from by omega]; exact Nat.mod_self _]
86         exact hv
87
88 /-- 环图 vs 路图二分: cycleGraph (m+3) 的邻接 = 路邻接 ∪ 环绕边 (val 口
89   径)。 -/
90 theorem cyc_path (m : ℕ) (u v : Fin (m+3)) :
91   (cycleGraph (m+3)).Adj u v ↔ (pathGraph (m+3)).Adj u v ∨
92     ((u : ℕ) = m + 2 ∧ (v : ℕ) = 0) ∨ ((u : ℕ) = 0 ∧ (v : ℕ) = m + 2) := by
93   rw [cycleGraph_adj_iff_succ, pathGraph_adj, fin_add_one_val m u v,
94     fin_add_one_val m v u]
95   constructor
96   · rintro ((h | ⟨h1, h2⟩) | (h | ⟨h1, h2⟩))
97     · exact Or.inl (Or.inl (by omega))
98     · exact Or.inr (Or.inl ⟨h1, h2⟩)
99     · exact Or.inl (Or.inr (by omega))
100    · exact Or.inr (Or.inr ⟨h2, h1⟩)
101  · rintro ((h | h) | ⟨h1, h2⟩ | ⟨h1, h2⟩)
102    · exact Or.inl (Or.inl (by omega))
103    · exact Or.inr (Or.inl (by omega))
104    · exact Or.inl (Or.inr ⟨h1, h2⟩)
105    · exact Or.inr (Or.inr ⟨h2, h1⟩)

```

```

103 theorem cyc_of_path (m : ℕ) (u v : Fin (m+3)) (h : (pathGraph (m+3)).Adj
    u v) :
104   (cycleGraph (m+3)).Adj u v := (cyc_path m u v).mpr (Or.inl h)
105
106 theorem cyc_wrap_0 (m : ℕ) :
107   (cycleGraph (m+3)).Adj ((m+2, by omega) : Fin (m+3)) ((0, by omega) :
    Fin (m+3)) := by
108   refine (cyc_path m _ _).mpr (Or.inr (Or.inl ⟨?, ?_⟩))
109   · exact rfl
110   · exact rfl
111
112 theorem cyc_wrap_1 (m : ℕ) :
113   (cycleGraph (m+3)).Adj ((0, by omega) : Fin (m+3)) ((m+2, by omega) :
    Fin (m+3)) := by
114   refine (cyc_path m _ _).mpr (Or.inr (Or.inr ⟨?, ?_⟩))
115   · exact rfl
116   · exact rfl
117
118 theorem top_val (m : ℕ) : ((m+2, by omega) : Fin (m+3)) : ℕ = m + 2 :=
    rfl
119 theorem zero_val (m : ℕ) : ((0, by omega) : Fin (m+3)) : ℕ = 0 := rfl
120
121 /-- 环绕 rim 边: 第 m+2 列 ~ 第 0 列, 行 j. -/
122 def wE (m : ℕ) (j : Fin 2) : Sym2 (Fin (m+3) × Fin 2) :=
123   s((m+2, by omega), j), ((0, by omega), j)
124
125 theorem f2_01 : (0 : Fin 2) ≠ (1 : Fin 2) := by decide
126
127 theorem wE_nondisj (m : ℕ) (j : Fin 2) :
128   ¬ Disjoint (edgeSupp (wE m j)) (edgeSupp (wE m j)) := by
129   intro h
130   have h4 := (disjoint_esupp_iff ((m+2, by omega), j) : Fin (m+3) × Fin
    2)
131     ((0, by omega), j) : Fin (m+3) × Fin 2)
132     ((m+2, by omega), j) : Fin (m+3) × Fin 2)
133     ((0, by omega), j) : Fin (m+3) × Fin 2)).mp h
134   exact h4.1 rfl
135
136 theorem wE_ne (m : ℕ) : wE m 0 ≠ wE m 1 := by
137   intro h
138   simp only [wE] at h
139   rw [Sym2.eq_iff] at h
140   rcases h with ⟨h1, h2⟩ | ⟨h1, h2⟩
141   · exact absurd (congrArg Prod.snd h1) f2_01
142   · have hf : ((m+2, by omega) : Fin (m+3)) = ((0, by omega) : Fin (m+3))
    :=
143     congrArg Prod.fst h1
144   have hv : ((m+2, by omega) : Fin (m+3)) : ℕ = ((0, by omega) : Fin
    (m+3)) : ℕ :=

```

```

145     congrArg Fin.val hf
146     rw [top_val, zero_val] at hv
147     omega
148
149 theorem wE_not_mem_Eset (m : ℕ) (j : Fin 2) : wE m j ∉ Eset (m+3) (m+3)
150 := by
151   intro h
152   have hadj := (eset_mem_mk ((⟨m+2, by omega⟩, j) : Fin (m+3) × Fin 2)
153     ((⟨0, by omega⟩, j) : Fin (m+3) × Fin 2)).mp h
154   have h1 := hadj.1
155   rw [lad_adj_iff] at h1
156   rcases h1 with ⟨hc, hr⟩ | ⟨hr, hc⟩
157   · have hA : (((⟨m+2, by omega⟩, j) : Fin (m+3) × Fin 2).1 : ℕ) = m + 2
158     := rfl
159   · have hB : (((⟨0, by omega⟩, j) : Fin (m+3) × Fin 2).1 : ℕ) = 0 := rfl
160   have hv := congrArg Fin.val hc
161   rw [hA, hB] at hv
162   omega
163   rcases hc with hc | hc
164   · have hA : (((⟨m+2, by omega⟩, j) : Fin (m+3) × Fin 2).1 : ℕ) = m + 2
165     := rfl
166   · have hB : (((⟨0, by omega⟩, j) : Fin (m+3) × Fin 2).1 : ℕ) = 0 :=
167     rfl
168   rw [hA, hB] at hc
169   omega
170
171 /-- Eset (m+3) (m+3) 的成员资格 = 梯子邻接 (列界自动成立)。 -/
172 theorem esetNN (m : ℕ) (a b : Fin (m+3) × Fin 2) :
173   s(a, b) ∈ Eset (m+3) (m+3) ↔ (pathGraph (m+3) ⊆ pathGraph 2).Adj a b
174 := by
175   rw [eset_mem_mk]
176   exact ⟨fun h => h.1, fun h => ⟨h, a.1.isLt, b.1.isLt⟩⟩
177
178 /-- Cst (m+3) (m+3) 的成员资格线性化 (r2 §1 口径)。 -/
179 theorem cstNN (m : ℕ) (a b : Fin (m+3) × Fin 2) :
180   s(a, b) ∈ PrismR2.Cst (m+3) (m+3) ↔ (pathGraph (m+3) ⊆ pathGraph 2).
181   Adj a b ∧
182   ((a.2 : ℕ) = 1 ∨ 1 ≤ (a.1 : ℕ) ∧ (a.1 : ℕ) + 2 ≤ m + 3) ∧
183   ((b.2 : ℕ) = 1 ∨ 1 ≤ (b.1 : ℕ) ∧ (b.1 : ℕ) + 2 ≤ m + 3) := by
184   simp only [PrismR2.Cst, Finset.mem_filter, PrismR2.filt2, esetNN]
185
186 /-- 与 wE 的 disjoint 条件 (列/行线性口径)。 -/
187 theorem disj_wE_iff (m : ℕ) (j : Fin 2) (a b : Fin (m+3) × Fin 2) :

```

```

186 Disjoint (edgeSupp (s(a, b))) (edgeSupp (wE m j)) ↔
187   ((a.1 : ℕ) ≠ m + 2 ∨ a.2 ≠ j) ∧ ((a.1 : ℕ) ≠ 0 ∨ a.2 ≠ j) ∧
188   ((b.1 : ℕ) ≠ m + 2 ∨ b.2 ≠ j) ∧ ((b.1 : ℕ) ≠ 0 ∨ b.2 ≠ j) := by
189 have h := disjoint_esupp_iff a b (⟨m+2, by omega⟩, j) (⟨0, by omega⟩, j)
190 constructor
191 · intro hd
192   obtain ⟨h1, h2, h3, h4⟩ := h.mp hd
193   refine ⟨?, ?, ?, ?⟩
194   · simpa only [top_val] using PrismR2.ne_to_col _ _ _ h1
195   · simpa only [zero_val] using PrismR2.ne_to_col _ _ _ h2
196   · simpa only [top_val] using PrismR2.ne_to_col _ _ _ h3
197   · simpa only [zero_val] using PrismR2.ne_to_col _ _ _ h4
198 · rintro ⟨h1, h2, h3, h4⟩
199   refine h.mpr ⟨?, ?, ?, ?⟩
200   · exact PrismR2.col_to_ne _ _ _ (by simpa only [top_val] using h1)
201   · exact PrismR2.col_to_ne _ _ _ (by simpa only [zero_val] using h2)
202   · exact PrismR2.col_to_ne _ _ _ (by simpa only [top_val] using h3)
203   · exact PrismR2.col_to_ne _ _ _ (by simpa only [zero_val] using h4)
204
205 /-! ## $Ob 三个结构映射: swapv (行翻转对合) / rotv (列旋转) / shEmb (列平
    移) -/
206
207 /-- 行翻转对合。 -/
208 def swapv {N : ℕ} (v : Fin N × Fin 2) : Fin N × Fin 2 := (v.1, 1 - v.2)
209
210 theorem swapv_inv {N : ℕ} (v : Fin N × Fin 2) : swapv (swapv v) = v :=
211   by
212     have hkey : (1 - (1 - v.2) : Fin 2) = v.2 := by
213       rcases fin2_ex v.2 with h | h <;> rw [h] <;> rfl
214     show (v.1, 1 - (1 - v.2)) = v
215     rw [hkey]
216
217 theorem swapv_inj {N : ℕ} : Function.Injective (swapv : Fin N × Fin 2 →
218   Fin N × Fin 2) := by
219   intro v v' h
220   have h2 : swapv (swapv v) = swapv (swapv v') := by rw [h]
221   rw [swapv_inv, swapv_inv] at h2
222   exact h2
223
224 theorem swapv_fst {N : ℕ} (v : Fin N × Fin 2) : (swapv v).1 = v.1 := rfl
225 theorem swapv_fst_val {N : ℕ} (v : Fin N × Fin 2) : ((swapv v).1 : ℕ) =
226   (v.1 : ℕ) := rfl
227 theorem swapv_snd_val {N : ℕ} (v : Fin N × Fin 2) : ((swapv v).2 : ℕ) =
228   1 - (v.2 : ℕ) :=
229   one_sub_val v.2
230
231 /-- 列旋转 (+1 mod m+3), 行不变。 -/
232 def rotv (m : ℕ) (v : Fin (m+3) × Fin 2) : Fin (m+3) × Fin 2 := (v.1 +
233   1, v.2)

```

```

229
230 theorem rotv_val_lt (m : ℕ) (v : Fin (m+3) × Fin 2) (h : (v.1 : ℕ) + 1 <
    m + 3) :
231   ((rotv m v).1 : ℕ) = (v.1 : ℕ) + 1 := by
232   show (((v.1 + 1 : Fin (m+3))) : ℕ) = (v.1 : ℕ) + 1
233   rw [Fin.val_add, Fin.val_one]
234   exact Nat.mod_eq_of_lt h
235
236 theorem rotv_val_top (m : ℕ) (v : Fin (m+3) × Fin 2) (h : (v.1 : ℕ) = m
    + 2) :
237   ((rotv m v).1 : ℕ) = 0 := by
238   show (((v.1 + 1 : Fin (m+3))) : ℕ) = 0
239   rw [Fin.val_add, Fin.val_one]
240   have hmod : ((v.1 : ℕ) + 1) % (m + 3) = 0 := by
241     rw [show (v.1 : ℕ) + 1 = m + 3 from by omega]
242     exact Nat.mod_self _
243   rw [hmod]
244
245 theorem rotv_row (m : ℕ) (v : Fin (m+3) × Fin 2) : (rotv m v).2 = v.2 :=
    rfl
246
247 theorem rotv_inj (m : ℕ) : Function.Injective (rotv m) := by
248   intro v v' h
249   have hrow : (rotv m v).2 = (rotv m v').2 := congrArg (fun p : Fin (m+3)
    × Fin 2 => p.2) h
250   have hcol0 : (rotv m v).1 = (rotv m v').1 := congrArg (fun p : Fin (m
    +3) × Fin 2 => p.1) h
251   have ha : (rotv m v).1 = v.1 + 1 := rfl
252   have hb : (rotv m v').1 = v'.1 + 1 := rfl
253   rw [ha, hb] at hcol0
254   have hval := congrArg Fin.val hcol0
255   rw [Fin.val_add, Fin.val_one, Fin.val_add, Fin.val_one] at hval
256   have hv : (v.1 : ℕ) < m + 3 := v.1.isLt
257   have hv' : (v'.1 : ℕ) < m + 3 := v'.1.isLt
258   have hcc : (v.1 : ℕ) = (v'.1 : ℕ) := by
259     rcases Nat.lt_or_ge ((v.1 : ℕ) + 1) (m + 3) with h1 | h1
260     · rcases Nat.lt_or_ge ((v'.1 : ℕ) + 1) (m + 3) with h2 | h2
261       · rw [Nat.mod_eq_of_lt h1, Nat.mod_eq_of_lt h2] at hval
262         omega
263       · rw [Nat.mod_eq_of_lt h1] at hval
264         have hmod : ((v'.1 : ℕ) + 1) % (m + 3) = 0 := by
265           rw [show (v'.1 : ℕ) + 1 = m + 3 from by omega]
266           exact Nat.mod_self _
267         rw [hmod] at hval
268         omega
269     · rcases Nat.lt_or_ge ((v'.1 : ℕ) + 1) (m + 3) with h2 | h2
270       · rw [Nat.mod_eq_of_lt h2] at hval
271         have hmod : ((v.1 : ℕ) + 1) % (m + 3) = 0 := by
272           rw [show (v.1 : ℕ) + 1 = m + 3 from by omega]

```

```

273     exact Nat.mod_self _
274     rw [hmod] at hval
275     omega
276     · have hmod1 : ((v.1 : ℕ) + 1) % (m + 3) = 0 := by
277       rw [show (v.1 : ℕ) + 1 = m + 3 from by omega]
278       exact Nat.mod_self _
279     have hmod2 : ((v'.1 : ℕ) + 1) % (m + 3) = 0 := by
280       rw [show (v'.1 : ℕ) + 1 = m + 3 from by omega]
281       exact Nat.mod_self _
282     rw [hmod1, hmod2] at hval
283     omega
284     exact Prod.ext_iff.mpr ⟨Fin.val_injective hcc, by exact hrow⟩
285
286 /-- 列平移嵌入 ( $col \mapsto col+1$ ) :  $Fin (m+1) \times Fin 2 \rightarrow Fin (m+3) \times Fin 2$ 。 -/
287 def shEmb (m : ℕ) (v : Fin (m+1) × Fin 2) : Fin (m+3) × Fin 2 :=
288   (⟨(v.1 : ℕ) + 1, by omega⟩, v.2)
289
290 theorem shEmb_inj (m : ℕ) : Function.Injective (shEmb m) := by
291   intro v v' h
292   have hrow : (shEmb m v).2 = (shEmb m v').2 := by rw [h]
293   have hcol : ((shEmb m v).1 : ℕ) = ((shEmb m v').1 : ℕ) := by rw [h]
294   have hA : ((shEmb m v).1 : ℕ) = (v.1 : ℕ) + 1 := rfl
295   have hB : ((shEmb m v').1 : ℕ) = (v'.1 : ℕ) + 1 := rfl
296   rw [hA, hB] at hcol
297   have hv : (v.1 : ℕ) < m + 1 := v.1.isLt
298   have hv' : (v'.1 : ℕ) < m + 1 := v'.1.isLt
299   exact Prod.ext_iff.mpr ⟨Fin.val_injective (by omega), by exact hrow⟩
300
301 theorem shEmb_val (m : ℕ) (v : Fin (m+1) × Fin 2) :
302   ((shEmb m v).1 : ℕ) = (v.1 : ℕ) + 1 := rfl
303 theorem shEmb_row (m : ℕ) (v : Fin (m+1) × Fin 2) : (shEmb m v).2 = v.2
304   := rfl
305
306 /-- emb 单射 ( $r1 transp$  证明体内嵌的核验件, 抽出复用)。 -/
307 theorem emb_inj {N k : ℕ} (hk : k ≤ N) : Function.Injective (emb hk) :=
308   by
309     intro v v' h
310     have h1 : (⟨(v.1 : ℕ), by omega⟩ : Fin N) = (⟨(v'.1 : ℕ), by omega⟩ :
311       Fin N) :=
312       congrArg Prod.fst h
313     have hv : v.1 = v'.1 := by
314       apply Fin.val_injective
315       show (v.1 : ℕ) = (v'.1 : ℕ)
316     exact congrArg (fun i : Fin N => (i : ℕ)) h1
317     have hs : (emb hk v).2 = (emb hk v').2 := congrArg (fun p : Fin N × Fin
318       2 => p.2) h
319     exact Prod.ext_iff.mpr ⟨hv, hs⟩
320

```

```

317 /-! ## S1 T4a: 棱柱边集两分类 (梯子边集 + 双环绕 rim 边) -/
318
319 theorem s_eq_wE_iff (m : ℕ) (j : Fin 2) (a b : Fin (m+3) × Fin 2) :
320   s(a, b) = wE m j ↔
321     (a = (⟨m+2, by omega⟩, j) ∧ b = (⟨0, by omega⟩, j)) ∨
322     (a = (⟨0, by omega⟩, j) ∧ b = (⟨m+2, by omega⟩, j)) := by
323   simp only [wE, Sym2.eq_iff]
324
325 theorem s_eq_wE_of (m : ℕ) (j : Fin 2) (a b : Fin (m+3) × Fin 2)
326   (ha : (a.1 : ℕ) = m + 2) (hb : (b.1 : ℕ) = 0) (hja : a.2 = j) (hjb :
327     b.2 = j) :
328   s(a, b) = wE m j := by
329   rw [s_eq_wE_iff]
330   exact Or.inl ⟨Prod.ext_iff.mpr ⟨Fin.val_injective ha, aja⟩,
331     Prod.ext_iff.mpr ⟨Fin.val_injective hb, hjb⟩⟩
332
333 theorem s_eq_wE_of' (m : ℕ) (j : Fin 2) (a b : Fin (m+3) × Fin 2)
334   (ha : (a.1 : ℕ) = 0) (hb : (b.1 : ℕ) = m + 2) (hja : a.2 = j) (hjb :
335     b.2 = j) :
336   s(a, b) = wE m j := by
337   rw [s_eq_wE_iff]
338   exact Or.inr ⟨Prod.ext_iff.mpr ⟨Fin.val_injective ha, aja⟩,
339     Prod.ext_iff.mpr ⟨Fin.val_injective hb, hjb⟩⟩
340
341 theorem wE_adj_prism (m : ℕ) (j : Fin 2) :
342   (prism (m+3)).Adj (⟨m+2, by omega⟩, j) (⟨0, by omega⟩, j) := by
343   show (cycleGraph (m+3) □ pathGraph 2).Adj (⟨m+2, by omega⟩, j) (⟨0, by
344     omega⟩, j)
345   exact boxProd_adj.mpr (Or.inl ⟨cyc_wrap_0 m, rfl⟩)
346
347 theorem wE_adj_prism' (m : ℕ) (j : Fin 2) :
348   (prism (m+3)).Adj (⟨0, by omega⟩, j) (⟨m+2, by omega⟩, j) := by
349   show (cycleGraph (m+3) □ pathGraph 2).Adj (⟨0, by omega⟩, j) (⟨m+2, by
350     omega⟩, j)
351   exact boxProd_adj.mpr (Or.inl ⟨cyc_wrap_1 m, rfl⟩)
352
353 theorem wE_mem_prism (m : ℕ) (j : Fin 2) : wE m j ∈ (prism (m+3)).
354   edgeFinset := by
355   simp only [wE]
356   rw [SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet]
357   exact wE_adj_prism m j
358
359 /-- **T4a**: 棱柱边集 = Eset (梯子) ∪ 双环绕。 -/
360 theorem prism_edges (m : ℕ) :
361   (prism (m+3)).edgeFinset = Eset (m+3) (m+3) ∪ {wE m 0, wE m 1} := by
362   ext e
363   induction e using Sym2.ind with
364   | _ a b =>
365   simp only [Finset.mem_union, Finset.mem_insert, Finset.mem_singleton,

```

```

    esetNN,
361   SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet]
362   have hb : (cycleGraph (m+3)  $\square$  pathGraph 2).Adj a b  $\leftrightarrow$ 
363     ((cycleGraph (m+3)).Adj a.1 b.1  $\wedge$  a.2 = b.2)  $\vee$ 
364     ((pathGraph 2).Adj a.2 b.2  $\wedge$  a.1 = b.1) := boxProd_adj
365   constructor
366   · intro h
367     have h' : (cycleGraph (m+3)  $\square$  pathGraph 2).Adj a b := h
368     rcases hb.mp h' with ⟨hcyc, hrow⟩ | ⟨hp2, hcol⟩
369     · rcases (cyc_path m a.1 b.1).mp hcyc with hpath | ⟨h1, h2⟩ | ⟨h1, h2⟩
370       · exact Or.inl (by rw [lad_adj_iff]; exact Or.inr ⟨hrow,
371         pathGraph_adj.mp hpath⟩)
372       · rcases fin2_ex a.2 with hr | hr
373         · exact Or.inr (Or.inl (s_eq_wE_of m 0 a b h1 h2 hr (hrow.symm.
374           trans hr)))
375         · exact Or.inr (Or.inl (s_eq_wE_of m 1 a b h1 h2 hr (hrow.symm.
376           trans hr)))
377       · exact Or.inl (by rw [lad_adj_iff]; exact Or.inl ⟨hcol, (p2adj a.2 b
378         .2).mp hp2⟩)
379     · rintro (hld | hw | hw)
380       · rcases (lad_adj_iff.mp hld) with ⟨hcol, hrow⟩ | ⟨hrow, hc⟩
381       · exact boxProd_adj.mpr (Or.inr ⟨(p2adj a.2 b.2).mpr hrow, hcol⟩)
382       · exact boxProd_adj.mpr (Or.inl ⟨cyc_of_path m a.1 b.1 (
383         pathGraph_adj.mpr hc), hrow⟩)
384     · rw [s_eq_wE_iff] at hw
385       · rcases hw with ⟨ha, hb2⟩ | ⟨ha, hb2⟩
386       · rw [ha, hb2]; exact wE_adj_prism m 0
387       · rw [ha, hb2]; exact wE_adj_prism' m 0
388     · rw [s_eq_wE_iff] at hw
389       · rcases hw with ⟨ha, hb2⟩ | ⟨ha, hb2⟩
390       · rw [ha, hb2]; exact wE_adj_prism m 1
391       · rw [ha, hb2]; exact wE_adj_prism' m 1
392   theorem hosoya_prism_mc (m : ℕ) :
393     hosoya (prism (m+3)) = matchCount (Eset (m+3) (m+3)  $\cup$  {wE m 0, wE m
394       1}) := by
395     rw [hosoya_matchCount, prism_edges]
396   /-! ## S2 T4b 四格 cell 引理 -/
397   theorem f2_val0 :  $\forall j : \text{Fin } 2, (j : \mathbb{N}) = 0 \rightarrow j = 0 := \text{by decide}$ 
398   theorem f2_val1 :  $\forall j : \text{Fin } 2, (j : \mathbb{N}) = 1 \rightarrow j = 1 := \text{by decide}$ 
399   theorem f2_val_cases :  $\forall j : \text{Fin } 2, (j : \mathbb{N}) = 0 \vee (j : \mathbb{N}) = 1 := \text{by}$ 
    decide

```

```

400 theorem f2_ne01 :  $\forall j : \text{Fin } 2, (j : \mathbb{N}) = 1 \rightarrow j \neq 0 := \text{by decide}$ 
401 theorem f2_ne10 :  $\forall j : \text{Fin } 2, (j : \mathbb{N}) = 0 \rightarrow j \neq 1 := \text{by decide}$ 
402 theorem one_sub_inj :  $\forall r s : \text{Fin } 2, (1 - r : \text{Fin } 2) = 1 - s \rightarrow r = s :=$ 
    by decide
403
404 theorem lad_adj_swap {N :  $\mathbb{N}$ } (x y :  $\text{Fin } N \times \text{Fin } 2$ ) :
405   (pathGraph N  $\sqcap$  pathGraph 2).Adj (swapv x) (swapv y)  $\leftrightarrow$ 
406   (pathGraph N  $\sqcap$  pathGraph 2).Adj x y := by
407   rw [lad_adj_iff, lad_adj_iff]
408   constructor
409   · rintro (<hcol, hr> | <hrow, hc>)
410     · exact Or.inl <hcol, fun h => hr (congrArg (fun j :  $\text{Fin } 2 \Rightarrow 1 - j$ ) h)>
411     · exact Or.inr <one_sub_inj x.2 y.2 hrow, hc>
412   · rintro (<hcol, hr> | <hrow, hc>)
413     · exact Or.inl <hcol, fun h => hr (one_sub_inj x.2 y.2 h)>
414     · exact Or.inr <congrArg (fun j :  $\text{Fin } 2 \Rightarrow 1 - j$ ) hrow, hc>
415
416 /-- **cell C2' (T4b-Eset 格) **: Eset 过滤 wEO-不相交格 = Cst. -/
417 theorem cell_wEO (m :  $\mathbb{N}$ ) :
418   (Eset (m+3) (m+3)).filter (fun f => Disjoint (edgeSupp f) (edgeSupp (wE m 0))) =
419     PrismR2.Cst (m+3) (m+3) := by
420   ext e
421   induction e using Sym2.ind with
422   | _ a b =>
423     simp only [Finset.mem_filter, esetNN, cstNN, disj_wE_iff m 0 a b]
424     have hais : (a.1 :  $\mathbb{N}$ ) < m + 3 := a.1.isLt
425     have hbis : (b.1 :  $\mathbb{N}$ ) < m + 3 := b.1.isLt
426     constructor
427     · rintro <hadj, h1, h2, h3, h4>
428       refine <hadj, ?, ?>
429       · rcases f2_val_cases a.2 with hva | hva
430         · refine Or.inr <?, ?>
431           · rcases h2 with h | h
432             · omega
433             · exact absurd (f2_val0 a.2 hva) h
434         · rcases h1 with h | h
435           · omega
436           · exact absurd (f2_val0 a.2 hva) h
437       · exact Or.inl hva
438     · rcases f2_val_cases b.2 with hvb | hvb
439       · refine Or.inr <?, ?>
440         · rcases h4 with h | h
441           · omega
442           · exact absurd (f2_val0 b.2 hvb) h
443         · rcases h3 with h | h
444           · omega
445         · exact absurd (f2_val0 b.2 hvb) h

```

```

446   · exact Or.inl hvb
447 · rintro ⟨hadj, hqa, hqb⟩
448   refine ⟨hadj, ?_, ?_, ?_, ?_⟩
449   · rcases f2_val_cases a.2 with hva | hva
450     · rcases hqa with hq | hq <;> omega
451     · exact Or.inr (f2_ne01 a.2 hva)
452   · rcases f2_val_cases a.2 with hva | hva
453     · rcases hqa with hq | hq <;> omega
454     · exact Or.inr (f2_ne01 a.2 hva)
455   · rcases f2_val_cases b.2 with hvb | hvb
456     · rcases hqb with hq | hq <;> omega
457     · exact Or.inr (f2_ne01 b.2 hvb)
458   · rcases f2_val_cases b.2 with hvb | hvb
459     · rcases hqb with hq | hq <;> omega
460     · exact Or.inr (f2_ne01 b.2 hvb)
461
462 theorem f2_flip_ne1 {N : ℕ} (a x : Fin N × Fin 2) (h : (a.2 : ℕ) = 1 - (
  x.2 : ℕ))
463   (hv : (x.2 : ℕ) = 1) : a.2 ≠ (1 : Fin 2) := by
464   intro hcon
465   have hva : (a.2 : ℕ) = 1 := by rw [hcon]; rfl
466   omega
467
468 theorem f2_flip_ne1' {N : ℕ} (a x : Fin N × Fin 2) (h : (a.2 : ℕ) = 1 -
  (x.2 : ℕ))
469   (hv : (x.2 : ℕ) = 0) : a.2 ≠ 0 := by
470   intro hcon
471   have hva : (a.2 : ℕ) = 0 := by rw [hcon]; rfl
472   omega
473
474 /-- **cell C1 (T4b-Eset 格) **: Eset 过滤 wE1-不相交格 = Cst 的行翻转像。
475   -/
476 theorem cell_wE1 (m : ℕ) :
477   (Eset (m+3) (m+3)).filter (fun f => Disjoint (edgeSupp f) (edgeSupp (
  wE m 1))) =
478   (PrismR2.Cst (m+3) (m+3)).image
479   (Sym2.map (swapv : Fin (m+3) × Fin 2 → Fin (m+3) × Fin 2)) :=
480   by
481   ext e
482   induction e using Sym2.ind with
483   | _ a b =>
484     simp only [Finset.mem_filter, Finset.mem_image, esetNN]
485     constructor
486   · intro h
487     have hadj := h.1
488     obtain ⟨h1, h2, h3, h4⟩ := (disj_wE_iff m 1 a b).mp h.2
489     refine ⟨s(swapv a, swapv b), ?_, ?_⟩
490     · rw [cstNN]
491     · refine ⟨(lad_adj_swap a b).mpr hadj, ?_, ?_⟩

```

```

490 · rcases f2_val_cases a.2 with hva | hva
491 · exact Or.inl (by have := swapv_snd_val a; omega)
492 · refine Or.inr ⟨?, ?⟩
493 · rcases h2 with h | h
494 · have := swapv_fst_val a; omega
495 · exact absurd (f2_val1 a.2 hva) h
496 · rcases h1 with h | h
497 · have := swapv_fst_val a; have := a.1.isLt; omega
498 · exact absurd (f2_val1 a.2 hva) h
499 · rcases f2_val_cases b.2 with hvb | hvb
500 · exact Or.inl (by have := swapv_snd_val b; omega)
501 · refine Or.inr ⟨?, ?⟩
502 · rcases h4 with h | h
503 · have := swapv_fst_val b; omega
504 · exact absurd (f2_val1 b.2 hvb) h
505 · rcases h3 with h | h
506 · have := swapv_fst_val b; have := b.1.isLt; omega
507 · exact absurd (f2_val1 b.2 hvb) h
508 · rw [Sym2.map_mk, swapv_inv, swapv_inv]
509 · rintro ⟨e', he', hmap⟩
510 · induction e' using Sym2.ind with
511 | _ x y =>
512   rw [Sym2.map_mk, Sym2.eq_iff] at hmap
513   obtain ⟨hadj, hqx, hqy⟩ := (cstNN m x y).mp he'
514   rcases hmap with ⟨hsa, hsb⟩ | ⟨hsb, hsa⟩
515   · have hA : a = swapv x := hsa.symm
516     have hB : b = swapv y := hsb.symm
517     subst hA
518     subst hB
519     have ha1 : ((swapv x).1 : ℕ) = (x.1 : ℕ) := swapv_fst_val x
520     have ha2 : ((swapv x).2 : ℕ) = 1 - (x.2 : ℕ) := swapv_snd_val x
521     have hb1 : ((swapv y).1 : ℕ) = (y.1 : ℕ) := swapv_fst_val y
522     have hb2 : ((swapv y).2 : ℕ) = 1 - (y.2 : ℕ) := swapv_snd_val y
523     refine ⟨(lad_adj_swap x y).mpr hadj, (disj_wE_iff m 1 (swapv x) (
524       swapv y)).mpr ⟨?, ?, ?, ?⟩⟩
525   · rcases f2_val_cases x.2 with hv | hv
526     · left; omega
527   · rcases f2_val_cases x.2 with hv | hv
528     · left; omega
529     · exact Or.inr (f2_flip_ne1 (swapv x) x ha2 hv)
530   · rcases f2_val_cases y.2 with hv | hv
531     · left; omega
532     · exact Or.inr (f2_flip_ne1 (swapv y) y hb2 hv)
533   · rcases f2_val_cases y.2 with hv | hv
534     · left; omega
535     · exact Or.inr (f2_flip_ne1 (swapv y) y hb2 hv)
536 · have hA : a = swapv y := hsa.symm
537   have hB : b = swapv x := hsb.symm

```

```

538     subst hA
539     subst hB
540     have ha1 : ((swapv y).1 : ℕ) = (y.1 : ℕ) := swapv_fst_val y
541     have ha2 : ((swapv y).2 : ℕ) = 1 - (y.2 : ℕ) := swapv_snd_val y
542     have hb1 : ((swapv x).1 : ℕ) = (x.1 : ℕ) := swapv_fst_val x
543     have hb2 : ((swapv x).2 : ℕ) = 1 - (x.2 : ℕ) := swapv_snd_val x
544     refine ⟨((lad_adj_swap x y).mpr hadj).symm, (disj_wE_iff m 1 (swapv
y) (swapv x)).mpr ⟨?, ?, ?, ?⟩⟩
545     · rcases f2_val_cases y.2 with hv | hv
546       · left; omega
547       · exact Or.inr (f2_flip_ne1 (swapv y) y ha2 hv)
548     · rcases f2_val_cases y.2 with hv | hv
549       · left; omega
550       · exact Or.inr (f2_flip_ne1 (swapv y) y ha2 hv)
551     · rcases f2_val_cases x.2 with hv | hv
552       · left; omega
553       · exact Or.inr (f2_flip_ne1 (swapv x) x hb2 hv)
554     · rcases f2_val_cases x.2 with hv | hv
555       · left; omega
556       · exact Or.inr (f2_flip_ne1 (swapv x) x hb2 hv)
557
558 theorem eset_full (N : ℕ) (a b : Fin N × Fin 2) :
559   s(a, b) ∈ Eset N N ↔ (pathGraph N ⊆ pathGraph 2).Adj a b := by
560   rw [eset_mem_mk]
561   exact ⟨fun h => h.1, fun h => ⟨h, a.1.isLt, b.1.isLt⟩⟩
562
563 /-- omega 断原子桥 (↑v.1 投影原子 omega 不直接连, 先抽象成纯 N 变量)。 -/
564 theorem sub_lt_aux (m c : ℕ) (h1 : 1 ≤ c) (h2 : c ≤ m + 1) : c - 1 < m +
1 := by omega
565
566 /-- 列前移一格 (col ↦ col-1) 的预像函数 (1 ≤ col ≤ m+1)。 -/
567 def vpred (m : ℕ) (v : Fin (m+3) × Fin 2) (h : 1 ≤ (v.1 : ℕ) ∧ (v.1 : ℕ
) ≤ m + 1) :
568   Fin (m+1) × Fin 2 :=
569   ((v.1 : ℕ) - 1, sub_lt_aux m (v.1 : ℕ) h.1 h.2), v.2)
570
571 theorem vpred_fst_val (m : ℕ) (v : Fin (m+3) × Fin 2) (h : 1 ≤ (v.1 : ℕ
) ∧ (v.1 : ℕ) ≤ m + 1) :
572   ((vpred m v h).1 : ℕ) = (v.1 : ℕ) - 1 := rfl
573 theorem vpred_snd (m : ℕ) (v : Fin (m+3) × Fin 2) (h : 1 ≤ (v.1 : ℕ) ∧
(v.1 : ℕ) ≤ m + 1) :
574   (vpred m v h).2 = v.2 := rfl
575
576 theorem cst_bound (m : ℕ) (v : Fin (m+3) × Fin 2)
577   (hq : (v.2 : ℕ) = 1 ∨ 1 ≤ (v.1 : ℕ) ∧ (v.1 : ℕ) + 2 ≤ m + 3)
578   (hc1 : (v.1 : ℕ) ≠ m + 2 ∨ v.2 ≠ (1 : Fin 2))
579   (hc2 : (v.1 : ℕ) ≠ 0 ∨ v.2 ≠ (1 : Fin 2)) :
580   1 ≤ (v.1 : ℕ) ∧ (v.1 : ℕ) ≤ m + 1 := by
581   have his := v.1.isLt

```

```

582 rcases f2_val_cases v.2 with hv | hv
583 · rcases hq with hq' | hq' <;> omega
584 · rcases hc1 with h | h
585   · rcases hc2 with h' | h'
586     · omega
587     · exact absurd (f2_val1 v.2 hv) h'
588   · exact absurd (f2_val1 v.2 hv) h
589
590 theorem lad_adj_pred (m : ℕ) (a b : Fin (m+3) × Fin 2)
591   (ha : 1 ≤ (a.1 : ℕ) ∧ (a.1 : ℕ) ≤ m + 1) (hb : 1 ≤ (b.1 : ℕ) ∧ (b
592     .1 : ℕ) ≤ m + 1) :
593   (pathGraph (m+1) ⊔ pathGraph 2).Adj (vpred m a ha) (vpred m b hb) ↔
594     (pathGraph (m+3) ⊔ pathGraph 2).Adj a b := by
595   rw [lad_adj_iff, lad_adj_iff]
596   have hxa : ((vpred m a ha).1 : ℕ) = (a.1 : ℕ) - 1 := rfl
597   have hxb : ((vpred m b hb).1 : ℕ) = (b.1 : ℕ) - 1 := rfl
598   constructor
599   · rintro (⟨hc, hr⟩ | ⟨hr, hc⟩)
600     · refine Or.inl ⟨?, hr⟩
601       apply Fin.val_injective
602       have hcv : (a.1 : ℕ) - 1 = (b.1 : ℕ) - 1 := congrArg Fin.val hc
603       omega
604     · refine Or.inr ⟨hr, ?_⟩
605       rcases hc with hc | hc
606       · left; omega
607       · right; omega
608   · rintro (⟨hc, hr⟩ | ⟨hr, hc⟩)
609     · refine Or.inl ⟨?, hr⟩
610       apply Fin.val_injective
611       have hcv : (a.1 : ℕ) = (b.1 : ℕ) := congrArg Fin.val hc
612       omega
613     · refine Or.inr ⟨hr, ?_⟩
614       rcases hc with hc | hc
615       · left; omega
616       · right; omega
617
618 theorem shEmb_vpred (m : ℕ) (v : Fin (m+3) × Fin 2) (h : 1 ≤ (v.1 : ℕ)
619   ∧ (v.1 : ℕ) ≤ m + 1) :
620   shEmb m (vpred m v h) = v := by
621   have hval : ((shEmb m (vpred m v h)).1 : ℕ) = (v.1 : ℕ) := by
622     show ((vpred m v h).1 : ℕ) + 1 = (v.1 : ℕ)
623     have hx : ((vpred m v h).1 : ℕ) = (v.1 : ℕ) - 1 := rfl
624     omega
625   exact Prod.ext_iff.mpr ⟨Fin.val_injective hval, rfl⟩
626
627 theorem shEmb_row_val (m : ℕ) (v : Fin (m+1) × Fin 2) : ((shEmb m v).2 :
628   ℕ) = (v.2 : ℕ) := rfl
629
630 theorem lad_adj_emb (m : ℕ) (x y : Fin (m+1) × Fin 2) :

```

```

628   (pathGraph (m+3)  $\sqcap$  pathGraph 2).Adj (shEmb m x) (shEmb m y)  $\leftrightarrow$ 
629   (pathGraph (m+1)  $\sqcap$  pathGraph 2).Adj x y := by
630   rw [lad_adj_iff, lad_adj_iff]
631   have hxa : ((shEmb m x).1 :  $\mathbb{N}$ ) = (x.1 :  $\mathbb{N}$ ) + 1 := rfl
632   have hxb : ((shEmb m y).1 :  $\mathbb{N}$ ) = (y.1 :  $\mathbb{N}$ ) + 1 := rfl
633   have hx : (x.1 :  $\mathbb{N}$ ) < m + 1 := x.1.isLt
634   have hy : (y.1 :  $\mathbb{N}$ ) < m + 1 := y.1.isLt
635   constructor
636   · rintro (<hc, hr> | <hr, hc>)
637     · refine Or.inl <?, hr>
638       apply Fin.val_injective
639       have hcv : ((shEmb m x).1 :  $\mathbb{N}$ ) = ((shEmb m y).1 :  $\mathbb{N}$ ) := congrArg
        Fin.val hc
640       omega
641     · refine Or.inr <hr, ?_>
642       rcases hc with hc | hc
643       · left; omega
644       · right; omega
645   · rintro (<hc, hr> | <hr, hc>)
646     · refine Or.inl <?, hr>
647       apply Fin.val_injective
648       have hcv : (x.1 :  $\mathbb{N}$ ) = (y.1 :  $\mathbb{N}$ ) := congrArg Fin.val hc
649       omega
650     · refine Or.inr <hr, ?_>
651       rcases hc with hc | hc
652       · left; omega
653       · right; omega
654
655   /-- **cell C3 (T4b-Cst 格) **: Cst 过滤 wE1-不相交格 = 列平移梯子像。 -/
656   theorem cell_C3 (m :  $\mathbb{N}$ ) :
657     (PrismR2.Cst (m+3) (m+3)).filter (fun f => Disjoint (edgeSupp f) (
658       edgeSupp (wE m 1))) =
659     (Eset (m+1) (m+1)).image (Sym2.map (shEmb m)) := by
660   ext e
661   induction e using Sym2.ind with
662   | _ a b =>
663   simp only [Finset.mem_filter, Finset.mem_image, cstNN]
664   constructor
665   · rintro (<hadj, hqa, hqb>, hdisj)
666     obtain <h1, h2, h3, h4> := (disj_wE_iff m 1 a b).mp hdisj
667     obtain <hal, har> := cst_bound m a hqa h1 h2
668     obtain <hbl, hbr> := cst_bound m b hqb h3 h4
669     refine <s(vpred m a <hal, har>, vpred m b <hbl, hbr>)>, ?_, ?_
670     · rw [eset_full]
671       exact (lad_adj_pred m a b <hal, har> <hbl, hbr>).mpr hadj
672     · rw [Sym2.map_mk, Sym2.eq_iff]
673       exact Or.inl <shEmb_vpred m a <hal, har>, shEmb_vpred m b <hbl, hbr>>
674   · rintro <e', he', hmap>
675     induction e' using Sym2.ind with

```

```

675 | _ x y =>
676 rw [Sym2.map_mk, Sym2.eq_iff] at hmap
677 have hadj := (eset_full (m+1) x y).mp he'
678 rcases hmap with ⟨hsa, hsb⟩ | ⟨hsb, hsa⟩
679 · have hA : a = shEmb m x := hsa.symm
680   have hB : b = shEmb m y := hsb.symm
681   subst hA
682   subst hB
683   have hxa : ((shEmb m x).1 : ℕ) = (x.1 : ℕ) + 1 := rfl
684   have hxb : ((shEmb m y).1 : ℕ) = (y.1 : ℕ) + 1 := rfl
685   have hx2 : ((shEmb m x).2 : ℕ) = (x.2 : ℕ) := rfl
686   have hy2 : ((shEmb m y).2 : ℕ) = (y.2 : ℕ) := rfl
687   have hx : (x.1 : ℕ) < m + 1 := x.1.isLt
688   have hy : (y.1 : ℕ) < m + 1 := y.1.isLt
689   refine ⟨((lad_adj_emb m x y).mpr hadj, ?_, ?_),
690     (disj_wE_iff m 1 (shEmb m x) (shEmb m y)).mpr ⟨?_, ?_, ?_, ?_⟩)
691   · rcases f2_val_cases x.2 with hv | hv
692     · refine Or.inr ⟨?_, ?_⟩
693       · omega
694       · omega
695     · exact Or.inl (by omega)
696   · rcases f2_val_cases y.2 with hv | hv
697     · refine Or.inr ⟨?_, ?_⟩
698       · omega
699       · omega
700     · exact Or.inl (by omega)
701   · rcases f2_val_cases x.2 with hv | hv
702     · exact Or.inr (f2_ne10 x.2 hv)
703     · left; omega
704   · rcases f2_val_cases x.2 with hv | hv
705     · exact Or.inr (f2_ne10 x.2 hv)
706     · left; omega
707   · rcases f2_val_cases y.2 with hv | hv
708     · exact Or.inr (f2_ne10 y.2 hv)
709     · left; omega
710   · rcases f2_val_cases y.2 with hv | hv
711     · exact Or.inr (f2_ne10 y.2 hv)
712     · left; omega
713 · have hA : a = shEmb m y := hsa.symm
714   have hB : b = shEmb m x := hsb.symm
715   subst hA
716   subst hB
717   have hxa : ((shEmb m x).1 : ℕ) = (x.1 : ℕ) + 1 := rfl
718   have hxb : ((shEmb m y).1 : ℕ) = (y.1 : ℕ) + 1 := rfl
719   have hx2 : ((shEmb m x).2 : ℕ) = (x.2 : ℕ) := rfl
720   have hy2 : ((shEmb m y).2 : ℕ) = (y.2 : ℕ) := rfl
721   have hx : (x.1 : ℕ) < m + 1 := x.1.isLt
722   have hy : (y.1 : ℕ) < m + 1 := y.1.isLt
723   refine ⟨((lad_adj_emb m x y).mpr hadj).symm, ?_, ?_),

```

```

724      (disj_wE_iff m 1 (shEmb m y) (shEmb m x)).mpr ⟨?, ?, ?, ?_⟩
725    · rcases f2_val_cases y.2 with hv | hv
726    · refine Or.inr ⟨?, ?_⟩
727      · omega
728      · omega
729    · exact Or.inl (by omega)
730    · rcases f2_val_cases x.2 with hv | hv
731    · refine Or.inr ⟨?, ?_⟩
732      · omega
733      · omega
734    · exact Or.inl (by omega)
735    · rcases f2_val_cases y.2 with hv | hv
736    · exact Or.inr (f2_ne10 y.2 hv)
737    · left; omega
738    · rcases f2_val_cases y.2 with hv | hv
739    · exact Or.inr (f2_ne10 y.2 hv)
740    · left; omega
741    · rcases f2_val_cases x.2 with hv | hv
742    · exact Or.inr (f2_ne10 x.2 hv)
743    · left; omega
744    · rcases f2_val_cases x.2 with hv | hv
745    · exact Or.inr (f2_ne10 x.2 hv)
746    · left; omega
747
748    /-! ## §3 T4b 组装:  $p = a + 2c + a(n-2)$  -/
749
750    theorem f2_10 : (1 : Fin 2) ≠ (0 : Fin 2) := fun h => f2_01 h.symm
751
752    /-- wE1 与 wE0 跨行不相交 (行 0 vs 行 1)。 -/
753    theorem wE_cross_disj (m : ℕ) : Disjoint (edgeSupp (wE m 1)) (edgeSupp (
754      wE m 0)) := by
755      have h := disjoint_esupp_iff ((⟨m+2, by omega⟩, (1 : Fin 2)) : Fin (m+3)
756        × Fin 2)
757        ((⟨0, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2)
758        ((⟨m+2, by omega⟩, (0 : Fin 2)) : Fin (m+3) × Fin 2)
759        ((⟨0, by omega⟩, (0 : Fin 2)) : Fin (m+3) × Fin 2)
760    refine h.mpr ⟨?, ?, ?, ?_⟩
761    · intro hcon; exact f2_10 (congrArg Prod.snd hcon)
762    · intro hcon
763      have hval := congrArg Fin.val (congrArg Prod.fst hcon)
764      have h1 : ((⟨m+2, by omega⟩ : Fin (m+3)) : ℕ) = m + 2 := rfl
765      have h2 : ((⟨0, by omega⟩ : Fin (m+3)) : ℕ) = 0 := rfl
766      rw [h1, h2] at hval
767      omega
768    · intro hcon
769      have hval := congrArg Fin.val (congrArg Prod.fst hcon)
770      have h1 : ((⟨0, by omega⟩ : Fin (m+3)) : ℕ) = 0 := rfl
771      have h2 : ((⟨m+2, by omega⟩ : Fin (m+3)) : ℕ) = m + 2 := rfl
772      rw [h1, h2] at hval

```

```

771     omega
772   · intro hcon; exact f2_10 (congrArg Prod.snd hcon)
773
774 theorem wE1_not_mem_Cst (m : ℕ) : wE m 1 ∉ PrismR2.Cst (m+3) (m+3) := by
775   intro h
776   simp only [PrismR2.Cst, Finset.mem_filter] at h
777   exact wE_not_mem_Eset m 1 h.1
778
779 /-- P \ {wEO} = Eset ∪ {wE1}. -/
780 theorem pE_sdiff (m : ℕ) :
781   (Eset (m+3) (m+3) ∪ {wE m 0, wE m 1}) \ {wE m 0} = Eset (m+3) (m+3) ∪
782     {wE m 1} := by
783   ext f
784   simp only [Finset.mem_sdiff, Finset.mem_union, Finset.mem_insert,
785     Finset.mem_singleton]
786   constructor
787   · rintro ⟨(h | h | h), hne⟩
788     · exact Or.inl h
789     · exact absurd h hne
790     · exact Or.inr h
791   · rintro (h | h)
792     · refine ⟨Or.inl h, ?_⟩
793       intro hcon
794       have hw : wE m 0 ∈ Eset (m+3) (m+3) := by rw [← hcon]; exact h
795       exact wE_not_mem_Eset m 0 hw
796     · refine ⟨Or.inr (Or.inr h), ?_⟩
797       intro hcon
798       exact wE_ne m (hcon.symm.trans h)
799
800 /-- (Eset ∪ {wE1}) \ {wE1} = Eset. -/
801 theorem eset_sdiff (m : ℕ) :
802   (Eset (m+3) (m+3) ∪ {wE m 1}) \ {wE m 1} = Eset (m+3) (m+3) := by
803   ext f
804   simp only [Finset.mem_sdiff, Finset.mem_union, Finset.mem_singleton]
805   constructor
806   · rintro ⟨(h | h), hne⟩
807     · exact h
808     · rw [h] at hne; exact absurd rfl hne
809   · intro h
810     refine ⟨Or.inl h, ?_⟩
811     intro hcon
812     have hw : wE m 1 ∈ Eset (m+3) (m+3) := by rw [← hcon]; exact h
813     exact wE_not_mem_Eset m 1 hw
814
815 /-- (Cst ∪ {wE1}) \ {wE1} = Cst. -/
816 theorem cst_sdiff (m : ℕ) :
817   (PrismR2.Cst (m+3) (m+3) ∪ {wE m 1}) \ {wE m 1} = PrismR2.Cst (m+3) (
818     m+3) := by
819   ext f

```

```

817 simp only [Finset.mem_sdiff, Finset.mem_union, Finset.mem_singleton]
818 constructor
819 · rintro ⟨(h | h), hne⟩
820 · exact h
821 · rw [h] at hne; exact absurd rfl hne
822 · intro h
823   refine ⟨Or.inl h, ?_⟩
824   intro hcon
825   have hw : wE m 1 ∈ PrismR2.Cst (m+3) (m+3) := by rw [← hcon]; exact
826     h
827   exact wE1_not_mem_Cst m hw
828 /-- (Eset ∪ {wE1}).filter (disj wE1) = Eset.filter (disj wE1). -/
829 theorem filter_u_wE1 (m : ℕ) :
830   (Eset (m+3) (m+3) ∪ {wE m 1}).filter (fun f => Disjoint (edgeSupp f)
831     (edgeSupp (wE m 1))) =
832     (Eset (m+3) (m+3)).filter (fun f => Disjoint (edgeSupp f) (edgeSupp
833       (wE m 1))) := by
834   ext f
835   simp only [Finset.mem_filter, Finset.mem_union, Finset.mem_singleton]
836   constructor
837   · rintro ⟨(h | h), hd⟩
838   · exact ⟨h, hd⟩
839   · rw [h] at hd; exact absurd hd (wE_nondisj m 1)
840   · exact fun h => ⟨Or.inl h.1, h.2⟩
841 /-- (Cst ∪ {wE1}).filter (disj wE1) = Cst.filter (disj wE1). -/
842 theorem filter_c_wE1 (m : ℕ) :
843   (PrismR2.Cst (m+3) (m+3) ∪ {wE m 1}).filter (fun f => Disjoint (
844     edgeSupp f) (edgeSupp (wE m 1))) =
845     (PrismR2.Cst (m+3) (m+3)).filter (fun f => Disjoint (edgeSupp f) (
846       edgeSupp (wE m 1))) := by
847   ext f
848   simp only [Finset.mem_filter, Finset.mem_union, Finset.mem_singleton]
849   constructor
850   · rintro ⟨(h | h), hd⟩
851   · exact ⟨h, hd⟩
852   · rw [h] at hd; exact absurd hd (wE_nondisj m 1)
853   · exact fun h => ⟨Or.inl h.1, h.2⟩
854 /-- {wE0, wE1}.filter (disj wE0) = {wE1}. -/
855 theorem filter_pair_wE0 (m : ℕ) :
856   ({wE m 0, wE m 1} : Finset (Sym2 (Fin (m+3) × Fin 2))).filter
857     (fun f => Disjoint (edgeSupp f) (edgeSupp (wE m 0))) = {wE m 1}
858   := by
859   ext f
860   simp only [Finset.mem_filter, Finset.mem_insert, Finset.mem_singleton]
861   constructor
862   · rintro ⟨(rfl | h), hd⟩

```

```

860   · exact absurd hd (wE_nondisj m 0)
861   · exact h
862   · intro h
863     rw [h]
864     exact ⟨Or.inr rfl, wE_cross_disj m⟩
865
866 /-- P.filter (disj wE0) = Cst ∪ {wE1} (cell C2) 。 -/
867 theorem pFilter (m : ℕ) :
868   (Eset (m+3) (m+3) ∪ {wE m 0, wE m 1}).filter (fun f => Disjoint (
869     edgeSupp f) (edgeSupp (wE m 0))) =
870     PrismR2.Cst (m+3) (m+3) ∪ {wE m 1} := by
871   rw [Finset.filter_union, cell_wE0, filter_pair_wE0]
872
873 /-- **T4b 第 2 步**:  $MC(Eset \cup \{wE1\}) = a(n) + c(n)$ 。 -/
874 theorem mc_union_wE1 (m : ℕ) :
875   matchCount (Eset (m+3) (m+3) ∪ {wE m 1}) =
876     hosoya (pathGraph (m+3) □ pathGraph 2) + matchCount (PrismR2.Cst (m
877       +3) (m+3)) := by
878   have hpart := matchCount_partition
879     (E := Eset (m+3) (m+3) ∪ {wE m 1}) (e := wE m 1) (by simp) (
880       wE_nondisj m 1)
881   rw [eset_sdiff m, filter_u_wE1 m] at hpart
882   have hE : matchCount (Eset (m+3) (m+3)) = hosoya (pathGraph (m+3) □
883     pathGraph 2) :=
884     (transp (le_refl (m+3))).symm
885   have hC : matchCount ((PrismR2.Cst (m+3) (m+3)).image
886     (Sym2.map (swapv : Fin (m+3) × Fin 2 → Fin (m+3) × Fin 2))) =
887     matchCount (PrismR2.Cst (m+3) (m+3)) :=
888     (matchCount_map_inj (swapv : Fin (m+3) × Fin 2 → Fin (m+3) × Fin 2)
889       swapv_inj _).symm
890   rw [hpart, cell_wE1 m, hE, hC]
891
892 /-- **T4b 第 5 步**:  $MC(Cst \cup \{wE1\}) = c(n) + a(n-2)$ 。 -/
893 theorem mc_Cst_wE1 (m : ℕ) :
894   matchCount (PrismR2.Cst (m+3) (m+3) ∪ {wE m 1}) =
895     matchCount (PrismR2.Cst (m+3) (m+3)) + hosoya (pathGraph (m+1) □
896     pathGraph 2) := by
897   have hpart := matchCount_partition
898     (E := PrismR2.Cst (m+3) (m+3) ∪ {wE m 1}) (e := wE m 1) (by simp) (
899       wE_nondisj m 1)
900   rw [cst_sdiff m, filter_c_wE1 m] at hpart
901   have hE : matchCount (Eset (m+1) (m+1)) = hosoya (pathGraph (m+1) □
902     pathGraph 2) :=
903     (transp (le_refl (m+1))).symm
904   have hC : matchCount ((Eset (m+1) (m+1)).image (Sym2.map (shEmb m))) =
905     matchCount (Eset (m+1) (m+1)) :=
906     (matchCount_map_inj (shEmb m) (shEmb_inj m) _).symm
907   rw [hpart, cell_C3 m, hC, hE]

```

```

901 /-- **T4b 主分解**:  $p(n) = a(n) + 2c(n) + a(n-2)$ 。 -/
902 theorem p_decomp (m : ℕ) :
903   matchCount (Eset (m+3) (m+3) ∪ {wE m 0, wE m 1}) =
904     hosoya (pathGraph (m+3) □ pathGraph 2) + 2 * matchCount (PrismR2.
905       Cst (m+3) (m+3)) +
906     hosoya (pathGraph (m+1) □ pathGraph 2) := by
907   have hpart := matchCount_partition
908     (E := Eset (m+3) (m+3) ∪ {wE m 0, wE m 1}) (e := wE m 0) (by simp) (
909       wE_nondisj m 0)
910   rw [pE_sdiff m, pFilter m, mc_union_wE1 m, mc_Cst_wE1 m] at hpart
911   omega
912
913 /-- Cst N m 的成员资格线性化 (任意 ambient)。 -/
914 theorem cstNN' {N m : ℕ} (a b : Fin N × Fin 2) :
915   s(a, b) ∈ PrismR2.Cst N m ↔ (pathGraph N □ pathGraph 2).Adj a b ∧
916     (a.1 : ℕ) < m ∧ (b.1 : ℕ) < m ∧
917     ((a.2 : ℕ) = 1 ∨ 1 ≤ (a.1 : ℕ) ∧ (a.1 : ℕ) + 2 ≤ m) ∧
918     ((b.2 : ℕ) = 1 ∨ 1 ≤ (b.1 : ℕ) ∧ (b.1 : ℕ) + 2 ≤ m) := by
919   simp only [PrismR2.Cst, Finset.mem_filter, PrismR2.filt2, eset_mem_mk,
920     and_assoc]
921
922 /-- emb 保 Cst 成员 (升 ambient)。 -/
923 theorem cst_emb_mem {N m : ℕ} (hk : m ≤ N) (x y : Fin m × Fin 2)
924   (hadj : (pathGraph m □ pathGraph 2).Adj x y) (hxm : (x.1 : ℕ) < m) (
925     hym : (y.1 : ℕ) < m)
926   (hqx : (x.2 : ℕ) = 1 ∨ 1 ≤ (x.1 : ℕ) ∧ (x.1 : ℕ) + 2 ≤ m)
927   (hqy : (y.2 : ℕ) = 1 ∨ 1 ≤ (y.1 : ℕ) ∧ (y.1 : ℕ) + 2 ≤ m) :
928   s(emb hk x, emb hk y) ∈ PrismR2.Cst N m := by
929   rw [cstNN']
930   refine ⟨?, hxm, hym, hqx, hqy⟩
931   rw [lad_adj_iff] at hadj ⊢
932   rcases hadj with ⟨hc, hr⟩ | ⟨hr, hc⟩
933   · refine Or.inl ⟨?, hr⟩
934     apply Fin.val_injective
935     have h1 : ((emb hk x).1 : ℕ) = (x.1 : ℕ) := rfl
936     have h2 : ((emb hk y).1 : ℕ) = (y.1 : ℕ) := rfl
937     rw [h1, h2]
938     exact congrArg Fin.val hc
939   · refine Or.inr ⟨hr, ?_⟩
940     have h1 : ((emb hk x).1 : ℕ) = (x.1 : ℕ) := rfl
941     have h2 : ((emb hk y).1 : ℕ) = (y.1 : ℕ) := rfl
942     rcases hc with hc | hc <;> omega
943
944 /-- **ambient 不变 (集合版)**:  $Cst\ N\ m = Cst\ m\ m$  的 emb 像。 -/
945 theorem cst_ambient_image (N m : ℕ) (hk : m ≤ N) :
946   PrismR2.Cst N m = (PrismR2.Cst m m).image (Sym2.map (emb hk)) := by
947   ext e

```

```

946 induction e using Sym2.ind with
947 | _ a b =>
948 rw [Finset.mem_image, cstNN']
949 constructor
950 · rintro ⟨hadj, hma, hmb, hqa, hqb⟩
951   have hax : emb hk (((a.1 : ℕ), by omega) : Fin m), a.2) = a := by
952     refine Prod.ext_iff.mpr ⟨?, rfl⟩
953     exact Fin.val_injective rfl
954   have hby : emb hk (((b.1 : ℕ), by omega) : Fin m), b.2) = b := by
955     refine Prod.ext_iff.mpr ⟨?, rfl⟩
956     exact Fin.val_injective rfl
957   refine ⟨s(((a.1 : ℕ), by omega) : Fin m), a.2), (((b.1 : ℕ), by
958     omega) : Fin m), b.2)),
959     ?_, ?_⟩
959   · rw [cstNN']
960     refine ⟨?, hma, hmb, hqa, hqb⟩
961     rw [lad_adj_iff] at hadj ⊢
962     rcases hadj with ⟨hc, hr⟩ | ⟨hr, hc⟩
963     · refine Or.inl ⟨?, hr⟩
964       apply Fin.val_injective
965       have h1 : (((a.1 : ℕ), by omega) : Fin m) : ℕ = (a.1 : ℕ) :=
966         rfl
967       have h2 : (((b.1 : ℕ), by omega) : Fin m) : ℕ = (b.1 : ℕ) :=
968         rfl
969       rw [h1, h2]
970       exact congrArg Fin.val hc
971     · refine Or.inr ⟨hr, ?_⟩
972       have h1 : (((a.1 : ℕ), by omega) : Fin m), a.2).1 : ℕ = (a.1 :
973         ℕ) := rfl
974       have h2 : (((b.1 : ℕ), by omega) : Fin m), b.2).1 : ℕ = (b.1 :
975         ℕ) := rfl
976       rcases hc with hc | hc <;> omega
977   · rw [Sym2.map_mk, hax, hby]
978   · rintro ⟨e', he', hmap⟩
979     induction e' using Sym2.ind with
980     | _ x y =>
981       rw [Sym2.map_mk] at hmap
982       rw [cstNN'] at he'
983       obtain ⟨hadj, hxm, hym, hqx, hqy⟩ := he'
984       rw [Sym2.eq_iff] at hmap
985       rcases hmap with ⟨h1, h2⟩ | ⟨h1, h2⟩
986       · rw [← h1, ← h2]
987         exact (cstNN' (emb hk x) (emb hk y)).mp (cst_emb_mem hk x y hadj
988           hxm hym hqx hqy)
989       · rw [← h2, ← h1]
990         exact (cstNN' (emb hk y) (emb hk x)).mp
991           (cst_emb_mem hk y x hadj.symm hym hxm hqy hqx)
992
993 /-- **ambient 不变 (计数版)**:  $MC(Cst\ N\ m) = MC(Cst\ m\ m)$ . -/

```

```

989 theorem cst_ambient (N m : ℕ) (hk : m ≤ N) :
990   matchCount (PrismR2.Cst N m) = matchCount (PrismR2.Cst m m) := by
991   rw [cst_ambient_image N m hk]
992   exact (matchCount_map_inj (emb hk) (emb_inj hk) _).symm
993
994 /-- **c-R4** (ambient-free 口径,  $n \geq 3$ )。 -/
995 theorem c_rec4' (n : ℕ) (hn : 3 ≤ n) :
996   matchCount (PrismR2.Cst (n+4) (n+4)) + matchCount (PrismR2.Cst n n) =
997     2 * matchCount (PrismR2.Cst (n+3) (n+3)) + 4 * matchCount (PrismR2.
998       Cst (n+2) (n+2)) := by
999   have h := PrismR2.c_rec4 (N := n+5) (n := n) hn (by omega)
1000   rw [cst_ambient (n+5) (n+4) (by omega), cst_ambient (n+5) n (by omega),
1001     cst_ambient (n+5) (n+3) (by omega), cst_ambient (n+5) (n+2) (by
1002       omega)] at h
1003   exact h
1004
1005 /-- **T4c**：棱柱主 R4。 -/
1006 theorem hosoya_prism_rec4 (m : ℕ) :
1007   hosoya (prism (m+7)) + hosoya (prism (m+3)) =
1008     2 * hosoya (prism (m+6)) + 4 * hosoya (prism (m+5)) := by
1009   have hm4 : hosoya (prism (m+7)) = matchCount (Eset (m+7) (m+7) ∪ {wE (m
1010     +4) 0, wE (m+4) 1}) :=
1011     hosoya_prism_mc (m+4)
1012   have hm3 : hosoya (prism (m+6)) = matchCount (Eset (m+6) (m+6) ∪ {wE (m
1013     +3) 0, wE (m+3) 1}) :=
1014     hosoya_prism_mc (m+3)
1015   have hm2 : hosoya (prism (m+5)) = matchCount (Eset (m+5) (m+5) ∪ {wE (m
1016     +2) 0, wE (m+2) 1}) :=
1017     hosoya_prism_mc (m+2)
1018   have hm0 : hosoya (prism (m+3)) = matchCount (Eset (m+3) (m+3) ∪ {wE m
1019     0, wE m 1}) :=
1020     hosoya_prism_mc m
1021   have pd4 : matchCount (Eset (m+7) (m+7) ∪ {wE (m+4) 0, wE (m+4) 1}) =
1022     hosoya (pathGraph (m+7) □ pathGraph 2) + 2 * matchCount (PrismR2.
1023       Cst (m+7) (m+7)) +
1024     hosoya (pathGraph (m+5) □ pathGraph 2) := p_decomp (m+4)
1025   have pd3 : matchCount (Eset (m+6) (m+6) ∪ {wE (m+3) 0, wE (m+3) 1}) =
1026     hosoya (pathGraph (m+6) □ pathGraph 2) + 2 * matchCount (PrismR2.
1027       Cst (m+6) (m+6)) +
1028     hosoya (pathGraph (m+4) □ pathGraph 2) := p_decomp (m+3)
1029   have pd2 : matchCount (Eset (m+5) (m+5) ∪ {wE (m+2) 0, wE (m+2) 1}) =
1030     hosoya (pathGraph (m+5) □ pathGraph 2) + 2 * matchCount (PrismR2.
1031       Cst (m+5) (m+5)) +
1032     hosoya (pathGraph (m+3) □ pathGraph 2) := p_decomp (m+2)
1033   have pd0 : matchCount (Eset (m+3) (m+3) ∪ {wE m 0, wE m 1}) =
1034     hosoya (pathGraph (m+3) □ pathGraph 2) + 2 * matchCount (PrismR2.
1035       Cst (m+3) (m+3)) +
1036     hosoya (pathGraph (m+1) □ pathGraph 2) := p_decomp m
1037   have ha0 : hosoya (pathGraph (m+4) □ pathGraph 2) + hosoya (pathGraph m

```

```

1028   □ pathGraph 2) =
1029     2 * hosoya (pathGraph (m+3) □ pathGraph 2) + 4 * hosoya (pathGraph
1030       (m+2) □ pathGraph 2) :=
1031     hosoya_ladder_rec4 m
1032   have ha1 : hosoya (pathGraph (m+5) □ pathGraph 2) + hosoya (pathGraph (
1033     m+1) □ pathGraph 2) =
1034     2 * hosoya (pathGraph (m+4) □ pathGraph 2) + 4 * hosoya (pathGraph
1035       (m+3) □ pathGraph 2) :=
1036     hosoya_ladder_rec4 (m+1)
1037   have ha2 : hosoya (pathGraph (m+6) □ pathGraph 2) + hosoya (pathGraph (
1038     m+2) □ pathGraph 2) =
1039     2 * hosoya (pathGraph (m+5) □ pathGraph 2) + 4 * hosoya (pathGraph
1040       (m+4) □ pathGraph 2) :=
1041     hosoya_ladder_rec4 (m+2)
1042   have ha3 : hosoya (pathGraph (m+7) □ pathGraph 2) + hosoya (pathGraph (
1043     m+3) □ pathGraph 2) =
1044     2 * hosoya (pathGraph (m+6) □ pathGraph 2) + 4 * hosoya (pathGraph
1045       (m+5) □ pathGraph 2) :=
1046     hosoya_ladder_rec4 (m+3)
1047   have hc : matchCount (PrismR2.Cst (m+7) (m+7)) + matchCount (PrismR2.
1048     Cst (m+3) (m+3)) =
1049     2 * matchCount (PrismR2.Cst (m+6) (m+6)) + 4 * matchCount (PrismR2.
1050       Cst (m+5) (m+5)) :=
1051     c_rec4' (m+3) (by omega)
1052   rw [hm4, hm0, hm3, hm2, pd4, pd0, pd3, pd2]
1053   omega
1054
1055   /-! ## $5 T4d rim 桥: 整体旋转把 delRim 边集映成 Eset ∪ {wE1} -/
1056
1057   /-- 无环绕时 Fin 加一的 val。 -/
1058   theorem fin_succ_val (m : ℕ) (i : Fin (m+3)) (h : (i : ℕ) + 1 < m + 3) :
1059     ((i + 1 : Fin (m+3)) : ℕ) = (i : ℕ) + 1 := by
1060     rw [Fin.val_add, Fin.val_one]
1061     exact Nat.mod_eq_of_lt h
1062
1063   /-- prismDelRim 删的那条 rim 边 (第 0-1 列、行 0)。 -/
1064   def e0 (m : ℕ) : Sym2 (Fin (m+3) × Fin 2) :=
1065     s((⟨0, by omega⟩, (0 : Fin 2)), (⟨1, by omega⟩, (0 : Fin 2)))
1066
1067   /-- 列旋转 -1 (含环绕), 行不变。 -/
1068   def colw (m : ℕ) (i : Fin (m+3)) : Fin (m+3) :=
1069     if (i : ℕ) = 0 then ⟨m+2, by omega⟩ else ⟨(i : ℕ) - 1, by omega⟩
1070
1071   def rotw (m : ℕ) (v : Fin (m+3) × Fin 2) : Fin (m+3) × Fin 2 := (colw m
1072     v.1, v.2)
1073
1074   theorem rotw_row (m : ℕ) (v : Fin (m+3) × Fin 2) : (rotw m v).2 = v.2 :=
1075     rfl

```

```

1065 theorem colw_val_zero (m : ℕ) (i : Fin (m+3)) (h : (i : ℕ) = 0) : ((colw
    m i) : ℕ) = m + 2 := by
1066   simp only [colw, if_pos h]
1067
1068 theorem colw_val_pos (m : ℕ) (i : Fin (m+3)) (h : 1 ≤ (i : ℕ)) : ((colw
    m i) : ℕ) = (i : ℕ) - 1 := by
1069   have hne : ¬ ((i : ℕ) = 0) := by omega
1070   simp only [colw, if_neg hne]
1071
1072 theorem colw_zero (m : ℕ) : colw m (⟨0, by omega⟩ : Fin (m+3)) = ⟨m+2, by
    omega⟩ := rfl
1073
1074 theorem colw_one (m : ℕ) : colw m (⟨1, by omega⟩ : Fin (m+3)) = ⟨0, by
    omega⟩ := by
1075   have h1 : ((⟨1, by omega⟩ : Fin (m+3)) : ℕ) = 1 := rfl
1076   have hne : ¬ (((⟨1, by omega⟩ : Fin (m+3)) : ℕ) = 0) := by
1077     intro hcon
1078     rw [h1] at hcon
1079     omega
1080   simp only [colw, if_neg hne]
1081
1082 theorem colw_inj (m : ℕ) : Function.Injective (colw m) := by
1083   intro i j h
1084   by_cases hi : (i : ℕ) = 0
1085   · by_cases hj : (j : ℕ) = 0
1086     · exact Fin.val_injective (by show (i : ℕ) = (j : ℕ); omega)
1087     · exfalso
1088       have hv := congrArg Fin.val h
1089       rw [colw_val_zero m i hi, colw_val_pos m j (by omega)] at hv
1090       have his : (i : ℕ) < m + 3 := i.isLt
1091       omega
1092   · by_cases hj : (j : ℕ) = 0
1093     · exfalso
1094       have hv := congrArg Fin.val h
1095       rw [colw_val_pos m i (by omega), colw_val_zero m j hj] at hv
1096       have his : (i : ℕ) < m + 3 := i.isLt
1097       omega
1098     · exact Fin.val_injective (by
1099       have hv := congrArg Fin.val h
1100       rw [colw_val_pos m i (by omega), colw_val_pos m j (by omega)] at
1101       hv
1102       omega)
1103
1104 theorem rotw_inj (m : ℕ) : Function.Injective (rotw m) := by
1105   intro v v' h
1106   have hrow : (rotw m v).2 = (rotw m v').2 := congrArg Prod.snd h
1107   have hcol : (rotw m v).1 = (rotw m v').1 := congrArg Prod.fst h
1108   exact Prod.ext_iff.mpr ⟨colw_inj m hcol, hrow⟩

```

```

1109 theorem rotv_rotw (m : ℕ) (v : Fin (m+3) × Fin 2) : rotv m (rotw m v) =
    v := by
1110   have hrow : (rotv m (rotw m v)).2 = v.2 := rfl
1111   have hc : (rotv m (rotw m v)).1 = v.1 := by
1112     show (colw m v.1) + 1 = v.1
1113     by_cases hv : (v.1 : ℕ) = 0
1114     · have hcw : ((colw m v.1) : ℕ) = m + 2 := colw_val_zero m v.1 hv
1115       apply Fin.val_injective
1116       rw [Fin.val_add, Fin.val_one, hcw]
1117       have hmod : (m + 2 + 1) % (m + 3) = 0 := by
1118         rw [show m + 2 + 1 = m + 3 from by omega]; exact Nat.mod_self _
1119       rw [hmod]
1120       omega
1121     · have hcw : ((colw m v.1) : ℕ) = (v.1 : ℕ) - 1 := colw_val_pos m v.1
1122       (by omega)
1123       have his : (v.1 : ℕ) < m + 3 := v.1.isLt
1124       apply Fin.val_injective
1125       rw [Fin.val_add, Fin.val_one, hcw]
1126       rw [Nat.mod_eq_of_lt (by omega)]
1127       omega
1128   exact Prod.ext_iff.mpr ⟨hc, hrow⟩
1129 theorem rotw_rotv (m : ℕ) (v : Fin (m+3) × Fin 2) : rotw m (rotv m v) =
    v := by
1130   have hrow : (rotw m (rotv m v)).2 = v.2 := rfl
1131   have hc : (rotw m (rotv m v)).1 = v.1 := by
1132     show colw m (v.1 + 1) = v.1
1133     by_cases hv : (v.1 : ℕ) = m + 2
1134     · have hvv : ((v.1 + 1 : Fin (m+3)) : ℕ) = 0 := by
1135       rw [Fin.val_add, Fin.val_one]
1136       have hmod : ((v.1 : ℕ) + 1) % (m + 3) = 0 := by
1137         rw [show (v.1 : ℕ) + 1 = m + 3 from by omega]; exact Nat.
1138         mod_self _
1139         rw [hmod]
1140       have hcw : ((colw m (v.1 + 1)) : ℕ) = m + 2 := colw_val_zero m (v.1
1141         + 1) hvv
1142       apply Fin.val_injective
1143       rw [hcw]
1144       omega
1145     · have hvv : ((v.1 + 1 : Fin (m+3)) : ℕ) = (v.1 : ℕ) + 1 :=
1146       fin_succ_val m v.1 (by omega)
1147       have hpos : 1 ≤ ((v.1 + 1 : Fin (m+3)) : ℕ) := by omega
1148       have hcw : ((colw m (v.1 + 1)) : ℕ) = (v.1 : ℕ) := by
1149         rw [colw_val_pos m (v.1 + 1) hpos, hvv]
1150         omega
1151       apply Fin.val_injective
1152       rw [hcw]
1153   exact Prod.ext_iff.mpr ⟨hc, hrow⟩

```

```

1153 /-- rotw 作用在 e0 上恰是 wE0。 -/
1154 theorem rotw_e0 (m : ℕ) : Sym2.map (rotw m) (e0 m) = wE m 0 := by
1155   simp only [e0, Sym2.map_mk]
1156   have h1 : rotw m ((⟨0, by omega⟩, (0 : Fin 2)) : Fin (m+3) × Fin 2) =
1157     ((⟨m+2, by omega⟩, (0 : Fin 2)) : Fin (m+3) × Fin 2) := by
1158     refine Prod.ext_iff.mpr ⟨?, rfl⟩
1159     show colw m (⟨0, by omega⟩ : Fin (m+3)) = _
1160     rw [colw_zero]
1161   have h2 : rotw m ((⟨1, by omega⟩, (0 : Fin 2)) : Fin (m+3) × Fin 2) =
1162     ((⟨0, by omega⟩, (0 : Fin 2)) : Fin (m+3) × Fin 2) := by
1163     refine Prod.ext_iff.mpr ⟨?, rfl⟩
1164     show colw m (⟨1, by omega⟩ : Fin (m+3)) = _
1165     rw [colw_one]
1166   rw [h1, h2]
1167   simp only [wE]
1168
1169 /-- rotv 把 wE1 映到第 0-1 列行 1 的梯子边。 -/
1170 theorem rotv_wE1 (m : ℕ) : Sym2.map (rotv m) (wE m 1) =
1171   s(((⟨0, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2),
1172     ((⟨1, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2)) := by
1173   simp only [wE, Sym2.map_mk]
1174   have h1 : rotv m ((⟨m+2, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2) =
1175     ((⟨0, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2) := by
1176     refine Prod.ext_iff.mpr ⟨?, rfl⟩
1177     apply Fin.val_injective
1178     show (((⟨m+2, by omega⟩ : Fin (m+3)) + 1 : Fin (m+3)) : ℕ) = 0
1179     rw [Fin.val_add, Fin.val_one, top_val m]
1180     have hmod : (m + 2 + 1) % (m + 3) = 0 := by
1181       rw [show m + 2 + 1 = m + 3 from by omega]; exact Nat.mod_self _
1182     rw [hmod]
1183   have h2 : rotv m ((⟨0, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2) =
1184     ((⟨1, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2) := by
1185     refine Prod.ext_iff.mpr ⟨?, rfl⟩
1186     apply Fin.val_injective
1187     show (((⟨0, by omega⟩ : Fin (m+3)) + 1 : Fin (m+3)) : ℕ) = 1
1188     rw [Fin.val_add, Fin.val_one, zero_val m]
1189     have hmod : (0 + 1) % (m + 3) = 1 := Nat.mod_eq_of_lt (by omega)
1190     rw [hmod]
1191   rw [h1, h2]
1192
1193 /-- 梯子邻接在 rotv 下的像仍是棱柱邻接。 -/
1194 theorem prism_adj_rotv (m : ℕ) (x y : Fin (m+3) × Fin 2)
1195   (hadj : (pathGraph (m+3) □ pathGraph 2).Adj x y) :
1196   (prism (m+3)).Adj (rotv m x) (rotv m y) := by
1197   show (cycleGraph (m+3) □ pathGraph 2).Adj (rotv m x) (rotv m y)
1198   rw [boxProd_adj]
1199   have his : (x.1 : ℕ) < m + 3 := x.1.isLt
1200   rw [lad_adj_iff] at hadj
1201   rcases hadj with ⟨hc, hr⟩ | ⟨hr, hc⟩

```

```

1202 · refine Or.inr ⟨(p2adj _ _).mpr hr, ?_⟩
1203   show (x.1 + 1 : Fin (m+3)) = (y.1 + 1 : Fin (m+3))
1204   exact congrArg (fun i : Fin (m+3) => i + 1) hc
1205 · refine Or.inl ⟨?_, hr⟩
1206   rw [cycleGraph_adj_iff_succ]
1207   rcases hc with hc | hc
1208 · refine Or.inl ?_
1209   have hfin : x.1 + 1 = y.1 := by
1210     apply Fin.val_injective
1211     rw [Fin.val_add, Fin.val_one]
1212     rw [show ((x.1 : ℕ) + 1) % (m + 3) = (x.1 : ℕ) + 1 from Nat.
mod_eq_of_lt (by omega)]
1213     exact hc
1214     show (y.1 + 1 : Fin (m+3)) = ((x.1 + 1 : Fin (m+3)) + 1)
1215     exact (congrArg (fun i : Fin (m+3) => i + 1) hfin).symm
1216 · refine Or.inr ?_
1217   have hfin : y.1 + 1 = x.1 := by
1218     apply Fin.val_injective
1219     rw [Fin.val_add, Fin.val_one]
1220     rw [show ((y.1 : ℕ) + 1) % (m + 3) = (y.1 : ℕ) + 1 from Nat.
mod_eq_of_lt (by omega)]
1221     exact hc
1222     show (x.1 + 1 : Fin (m+3)) = ((y.1 + 1 : Fin (m+3)) + 1)
1223     exact (congrArg (fun i : Fin (m+3) => i + 1) hfin).symm
1224
1225 /-- 第 0-1 列行 1 的梯子边是棱柱边。 -/
1226 theorem adj01 (m : ℕ) :
1227   (prism (m+3)).Adj ((⟨0, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2)
1228   ((⟨1, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2) := by
1229   show (cycleGraph (m+3) □ pathGraph 2).Adj _ _
1230   refine boxProd_adj.mpr (Or.inl ⟨?_, rfl⟩)
1231   rw [cycleGraph_adj_iff_succ]
1232   refine Or.inl ?_
1233   apply Fin.val_injective
1234   have hA : ((⟨1, by omega⟩ : Fin (m+3)) : ℕ) = 1 := rfl
1235   have hB : ((⟨0, by omega⟩ : Fin (m+3)) : ℕ) = 0 := rfl
1236   rw [Fin.val_add, Fin.val_one, hA, hB]
1237   exact (Nat.mod_eq_of_lt (by omega)).symm
1238
1239 /-- 行 1 的边不是 e0。 -/
1240 theorem ne01 (m : ℕ) :
1241   s(((⟨0, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2),
1242     ((⟨1, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2)) ≠ e0 m := by
1243   intro hcon
1244   simp only [e0] at hcon
1245   rw [Sym2.eq_iff] at hcon
1246   rcases hcon with ⟨h1, h2⟩ | ⟨h1, h2⟩
1247   · exact f2_10 (congrArg Prod.snd h1)
1248   · exact f2_10 (congrArg Prod.snd h2)

```

```

1249
1250 theorem sym2_swap (V : Type*) (x y : V) : s(x, y) = s(y, x) :=
1251   (Sym2.eq_iff.mpr (Or.inr ⟨rfl, rfl⟩))
1252
1253 /-- **cell C4 (旋转版) **: (Eset  $\cup$  {wE1}) 的 rotv 像 = prismDelRim 边集。
1254   -/
1254 theorem cell_C4 (m : ℕ) :
1255   (Eset (m+3) (m+3)  $\cup$  {wE m 1}).image (Sym2.map (rotv m)) =
1256   (prism (m+3)).edgeFinset \ {e0 m} := by
1257     ext e
1258     induction e using Sym2.ind with
1259       | _ a b =>
1260       simp only [Finset.mem_image, Finset.mem_sdiff, Finset.mem_union, Finset
1261         .mem_singleton,
1262         SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet]
1263     constructor
1264     · rintro ⟨e', hmem, hmap⟩
1265       induction e' using Sym2.ind with
1266         | _ x y =>
1267         rw [Sym2.map_mk] at hmap
1268         rcases hmem with hE | hW
1269         · have hadj : (prism (m+3)).Adj (rotv m x) (rotv m y) :=
1270           prism_adj_rotv m x y ((esetNN m x y).mp hE)
1271         have hne : s(rotv m x, rotv m y)  $\neq$  e0 m := by
1272           intro hcon
1273           have hc2 : Sym2.map (rotw m) (s(rotv m x, rotv m y)) = Sym2.map (
1274             rotw m) (e0 m) :=
1275             congrArg (Sym2.map (rotw m)) hcon
1276           rw [Sym2.map_mk, rotw_rotv, rotw_rotv, rotw_e0] at hc2
1277           exact wE_not_mem_Eset m 0 (by rw [← hc2]; exact hE)
1278         rcases (Sym2.eq_iff.mp hmap) with ⟨h1, h2⟩ | ⟨h1, h2⟩
1279         · rw [← h1, ← h2]
1280           exact ⟨hadj, hne⟩
1281         · rw [← h2, ← h1]
1282           exact ⟨hadj.symm,
1283             fun hcon => hne ((sym2_swap _ (rotv m x) (rotv m y)).trans hcon
1284               )⟩
1285     · have hmma : Sym2.map (rotv m) (s(x, y)) = s(rotv m x, rotv m y) :=
1286       Sym2.map_mk (rotv m) x y
1287       rw [hw, rotv_wE1] at hmma
1288       have hab := hmma.trans hmap
1289       rw [Sym2.eq_iff] at hab
1290       rcases hab with ⟨h1, h2⟩ | ⟨h1, h2⟩
1291       · rw [← h1, ← h2]
1292         exact ⟨adj01 m, ne01 m⟩
1293       · rw [← h1, ← h2]
1294         exact ⟨(adj01 m).symm, fun hcon =>
1295           ne01 m ((sym2_swap _
1296             ((⟨0, by omega⟩, (1 : Fin 2)) : Fin (m+3)  $\times$  Fin 2)

```

```

1293      ((⟨1, by omega⟩, (1 : Fin 2)) : Fin (m+3) × Fin 2)).trans
      hcon)⟩
1294 · rintro ⟨hadj, hne⟩
1295   refine ⟨s(rotw m a, rotw m b), ?_, ?_⟩
1296 · rw [show (prism (m+3)).Adj a b = ((cycleGraph (m+3) □ pathGraph 2).
Adj a b) from rfl] at hadj
1297   rw [boxProd_adj] at hadj
1298   rcases hadj with ⟨hcyc, hrow⟩ | ⟨hp2, hcol⟩
1299 · rw [cycleGraph_adj_iff_succ] at hcyc
1300   rcases f2_val_cases a.2 with hv | hv
1301 · -- 行 0: a.2 = 0
1302   by_cases ha0 : (a.1 : ℕ) = 0
1303 · rcases hcyc with hb | hb
1304 · -- b.1 = a.1+1 = ⟨1⟩ → 与 ≠0 矛盾
1305   exfalse
1306   apply hne
1307   have hvt : a.1 = (⟨0, by omega⟩ : Fin (m+3)) :=
1308     Fin.val_injective (by show (a.1 : ℕ) = 0; omega)
1309   have hbv : (b.1 : ℕ) = 1 := by
1310     rcases (fin_add_one_val m a.1 b.1).mp hb with h | ⟨h, h2⟩
1311     · omega
1312     · omega
1313   have hbt : b.1 = (⟨1, by omega⟩ : Fin (m+3)) :=
1314     Fin.val_injective (by show (b.1 : ℕ) = 1; omega)
1315   have hrowb : b.2 = (0 : Fin 2) := by
1316     rw [← hrow]; exact f2_val0 a.2 hv
1317   have hat : a = ((⟨0, by omega⟩, (0 : Fin 2)) : Fin (m+3) ×
Fin 2) :=
1318     Prod.ext_iff.mpr ⟨hvt, f2_val0 a.2 hv⟩
1319   have hbt2 : b = ((⟨1, by omega⟩, (0 : Fin 2)) : Fin (m+3) ×
Fin 2) :=
1320     Prod.ext_iff.mpr ⟨hbt, hrowb⟩
1321   rw [hat, hbt2]
1322   rfl
1323 · -- a.1 = b.1+1, b.1.val = m+2: 梯子边
1324   refine Or.inl ?_
1325   rw [esetNN, lad_adj_iff]
1326   refine Or.inr ⟨hrow, ?_⟩
1327   have hbv : (b.1 : ℕ) = m + 2 := by
1328     rcases (fin_add_one_val m b.1 a.1).mp hb with h | ⟨h, h2⟩
1329     · omega
1330     · omega
1331   have hca : ((rotw m a).1 : ℕ) = m + 2 := colw_val_zero m a
.1 ha0
1332   have hcb : ((rotw m b).1 : ℕ) = (b.1 : ℕ) - 1 :=
colw_val_pos m b.1 (by omega)
1333   rcases f2_val_cases b.2 with hvb | hvb
1334   · right; omega
1335   · right; omega

```

```

1336 · by_cases hb0 : (b.1 : ℕ) = 0
1337 · rcases hcyc with hb | hb
1338 · -- b.1 = a.1+1 (a.1.val = m+2 环绕) : 梯子边
1339   refine Or.inl ?_
1340   rw [esetNN, lad_adj_iff]
1341   refine Or.inr ⟨hrow, ?_⟩
1342   have hav : (a.1 : ℕ) = m + 2 := by
1343     rcases (fin_add_one_val m a.1 b.1).mp hb with h | ⟨h, h2
1344   ⟩
1345     · omega
1346     · omega
1347   have hca : ((rotrw m a).1 : ℕ) = (a.1 : ℕ) - 1 :=
1348     colw_val_pos m a.1 (by omega)
1349   have hcb : ((rotrw m b).1 : ℕ) = m + 2 := colw_val_zero m
1350     b.1 hb0
1351   rcases f2_val_cases b.2 with hvb | hvb
1352   · left; omega
1353   · left; omega
1354   · -- a.1 = b.1+1 = ⟨1⟩ → 与 ≠e0 矛盾
1355     exfalse
1356     apply hne
1357     have hbt : b.1 = ⟨0, by omega⟩ : Fin (m+3)) :=
1358       Fin.val_injective (by show (b.1 : ℕ) = 0; omega)
1359     have hav : (a.1 : ℕ) = 1 := by
1360       rcases (fin_add_one_val m b.1 a.1).mp hb with h | ⟨h, h2
1361     ⟩
1362       · omega
1363       · omega
1364     have hat : a.1 = ⟨1, by omega⟩ : Fin (m+3)) :=
1365       Fin.val_injective (by show (a.1 : ℕ) = 1; omega)
1366     have hrowa : a.2 = (0 : Fin 2) := f2_val0 a.2 hv
1367     have hrowb : b.2 = (0 : Fin 2) := by
1368       rw [← hrow]; exact hrowa
1369     have hat2 : a = (⟨1, by omega⟩, (0 : Fin 2)) : Fin (m+
1370     3) × Fin 2 :=
1371       Prod.ext_iff.mpr ⟨hat, hrowa⟩
1372     have hbt2 : b = (⟨0, by omega⟩, (0 : Fin 2)) : Fin (m+3)
1373     × Fin 2 :=
1374       Prod.ext_iff.mpr ⟨hbt, hrowb⟩
1375     rw [hat2, hbt2]
1376     exact sym2_swap _ (⟨1, by omega⟩, (0 : Fin 2)) : Fin (m
1377     +3) × Fin 2)
1378     (⟨0, by omega⟩, (0 : Fin 2)) : Fin (m+3) × Fin 2)
1379   · -- 两侧 val ≥ 1: colw = -val1, 梯子邻接
1380     refine Or.inl ?_
1381     rw [esetNN, lad_adj_iff]
1382     refine Or.inr ⟨hrow, ?_⟩
1383     have hca : ((rotrw m a).1 : ℕ) = (a.1 : ℕ) - 1 :=
1384       colw_val_pos m a.1 (by omega)

```

```

1377         have hcb : ((rotw m b).1 : ℕ) = (b.1 : ℕ) - 1 :=
colw_val_pos m b.1 (by omega)
1378     rcases hcyc with hb | hb
1379     · rcases (fin_add_one_val m a.1 b.1).mp hb with h | ⟨h, h2⟩
1380       · left; omega
1381       · omega
1382     · rcases (fin_add_one_val m b.1 a.1).mp hb with h | ⟨h, h2⟩
1383       · right; omega
1384       · omega
1385     · -- 行 1: a.2 = 1
1386       by_cases ha0 : (a.1 : ℕ) = 0
1387     · rcases hcyc with hb | hb
1388       · -- b.1 = ⟨1⟩: 像恰是 wE1
1389         refine Or.inr ?_
1390         have hvt : a.1 = ⟨0, by omega⟩ : Fin (m+3) :=
1391           Fin.val_injective (by show (a.1 : ℕ) = 0; omega)
1392         have hbv : (b.1 : ℕ) = 1 := by
1393           rcases (fin_add_one_val m a.1 b.1).mp hb with h | ⟨h, h2⟩
1394           · omega
1395           · omega
1396         have hbt : b.1 = ⟨1, by omega⟩ : Fin (m+3) :=
1397           Fin.val_injective (by show (b.1 : ℕ) = 1; omega)
1398         have hrowa : a.2 = (1 : Fin 2) := f2_val1 a.2 hv
1399         have hrowb : b.2 = (1 : Fin 2) := by rw [← hrow]; exact
hrowa
1400         have ha : rotw m a = (⟨m+2, by omega⟩, (1 : Fin 2)) : Fin (
m+3) × Fin 2 := by
1401           refine Prod.ext_iff.mpr ⟨?_, hrowa⟩
1402           show colw m a.1 = _
1403           rw [hvt, colw_zero]
1404         have hb : rotw m b = (⟨0, by omega⟩, (1 : Fin 2)) : Fin (m
+3) × Fin 2 := by
1405           refine Prod.ext_iff.mpr ⟨?_, hrowb⟩
1406           show colw m b.1 = _
1407           rw [hbt, colw_one]
1408         rw [ha, hb]
1409         simp only [wE]
1410       · -- a.1 = b.1+1, b.1.val = m+2: 梯子边
1411         refine Or.inl ?_
1412         rw [esetNN, lad_adj_iff]
1413         refine Or.inr ⟨hrow, ?_⟩
1414         have hbv : (b.1 : ℕ) = m + 2 := by
1415           rcases (fin_add_one_val m b.1 a.1).mp hb with h | ⟨h, h2⟩
1416           · omega
1417           · omega
1418         have hca : ((rotw m a).1 : ℕ) = m + 2 := colw_val_zero m a
.1 ha0
1419         have hcb : ((rotw m b).1 : ℕ) = (b.1 : ℕ) - 1 :=
colw_val_pos m b.1 (by omega)

```

```

1420         rcases f2_val_cases b.2 with hvb | hvb
1421         · right; omega
1422         · right; omega
1423     · by_cases hb0 : (b.1 : ℕ) = 0
1424     · rcases hcyc with hb | hb
1425       · -- b.1 = a.1+1 (a.1.val = m+2 环绕): 梯子边
1426         refine Or.inl ?_
1427         rw [esetNN, lad_adj_iff]
1428         refine Or.inr ⟨hrow, ?_⟩
1429         have hav : (a.1 : ℕ) = m + 2 := by
1430           rcases (fin_add_one_val m a.1 b.1).mp hb with h | ⟨h, h2
1431         · omega
1432         · omega
1433         have hca : ((rotw m a).1 : ℕ) = (a.1 : ℕ) - 1 :=
1434 colw_val_pos m a.1 (by omega)
1435         have hcb : ((rotw m b).1 : ℕ) = m + 2 := colw_val_zero m
1436 b.1 hb0
1437         rcases f2_val_cases b.2 with hvb | hvb
1438         · left; omega
1439         · left; omega
1440       · -- a.1 = b.1+1 = ⟨1⟩: 像恰是 wE1 (对称)
1441         refine Or.inr ?_
1442         have hbt : b.1 = ⟨0, by omega⟩ : Fin (m+3)) :=
1443           Fin.val_injective (by show (b.1 : ℕ) = 0; omega)
1444         have hav : (a.1 : ℕ) = 1 := by
1445           rcases (fin_add_one_val m b.1 a.1).mp hb with h | ⟨h, h2
1446         · omega
1447         · omega
1448         have hat : a.1 = ⟨1, by omega⟩ : Fin (m+3)) :=
1449           Fin.val_injective (by show (a.1 : ℕ) = 1; omega)
1450         have hrowa : a.2 = (1 : Fin 2) := f2_val1 a.2 hv
1451         have hrowb : b.2 = (1 : Fin 2) := by rw [← hrow]; exact
1452 hrowa
1453         have ha : rotw m a = (⟨0, by omega⟩, (1 : Fin 2)) : Fin (
1454 m+3) × Fin 2) := by
1455           refine Prod.ext_iff.mpr ⟨?_, hrowa⟩
1456           show colw m a.1 = _
1457           rw [hat, colw_one]
1458         have hb : rotw m b = (⟨m+2, by omega⟩, (1 : Fin 2)) : Fin
1459 (m+3) × Fin 2) := by
1460           refine Prod.ext_iff.mpr ⟨?_, hrowb⟩
1461           show colw m b.1 = _
1462           rw [hbt, colw_zero]
1463         rw [ha, hb]
1464         simp only [wE]
1465         exact sym2_swap _ _ _
1466       · -- 两侧 val ≥ 1: 梯子邻接

```

```

1462         refine Or.inl ?_
1463         rw [esetNN, lad_adj_iff]
1464         refine Or.inr ⟨hrow, ?_⟩
1465         have hca : ((rotrw m a).1 : ℕ) = (a.1 : ℕ) - 1 :=
colw_val_pos m a.1 (by omega)
1466         have hcb : ((rotrw m b).1 : ℕ) = (b.1 : ℕ) - 1 :=
colw_val_pos m b.1 (by omega)
1467         rcases hcyc with hb | hb
1468         · rcases (fin_add_one_val m a.1 b.1).mp hb with h | ⟨h, h2⟩
1469           · left; omega
1470           · omega
1471         · rcases (fin_add_one_val m b.1 a.1).mp hb with h | ⟨h, h2⟩
1472           · right; omega
1473           · omega
1474         · -- 竖档: 行翻转、列不变
1475           refine Or.inl ?_
1476           rw [esetNN, lad_adj_iff]
1477           refine Or.inl ⟨congrArg (colw m) hcol, (p2adj _ _).mp hp2⟩
1478           · rw [Sym2.map_mk, rotrw_rotrw, rotrw_rotrw]
1479
1480 /-- prismDelRim 的边集 = 棱柱边集 \ {e0}。 -/
1481 theorem delrim_edges (m : ℕ) (hm : 3 ≤ m + 3) :
1482   (prismDelRim (m+3) hm).edgeFinset = (prism (m+3)).edgeFinset \ {e0 m}
:= by
1483   ext f
1484   induction f using Sym2.ind with
1485   | _ x y =>
1486     simp only [Finset.mem_sdiff, Finset.mem_singleton]
1487     rw [SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet,
1488       SimpleGraph.mem_edgeFinset, SimpleGraph.mem_edgeSet]
1489     simp only [prismDelRim]
1490     rw [SimpleGraph.deleteEdges_adj, Set.mem_singleton_iff]
1491     simp only [e0]
1492     rfl
1493
1494 /-- **T4d**:  $d_{rim}(n) = a(n) + c(n)$ 。 -/
1495 theorem delrim_eq (m : ℕ) (hm : 3 ≤ m + 3) :
1496   hosoya (prismDelRim (m+3) hm) =
1497     hosoya (pathGraph (m+3) □ pathGraph 2) + matchCount (PrismR2.Cst (m
+3) (m+3)) := by
1498   rw [hosoya_matchCount, delrim_edges m hm, ← cell_C4]
1499   rw [← (matchCount_map_inj (rotrw m) (rotrw_inj m) (Eset (m+3) (m+3) ∪ {
wE m 1})))]
1500   exact mc_union_wE1 m
1501
1502 /-! ## S6 T4e spoke 桥 + T4f 初值/强归纳 + 主桥收口
1503
1504 探针 provenance: T4e = 子代理 agent-15/18 (双批同参, 探针 probe-t4e.lean
EXIT=0) ;

```

```

1505 初值 = agent-16/19 (probe-t4f-init.lean, c6 经库件 T1-T7 递推链, decide 超
    界留档);
1506 强归纳/R4 = agent-17/20 (probe-t4f-ind.lean)。组装 = 主代理 scaffold。
1507 -/
1508
1509 set_option maxRecDepth 40000
1510 set_option maxHeartbeats 1000000
1511
1512 -- ===== T4e spoke 桥 =====
1513
1514 /-- **T4e**: d_spoke 分割恒等式——删 spoke 的棱柱匹配数 = 完整棱柱 - (m+2)
    梯子。 -/
1515 theorem delspoke_eq (m : ℕ) (hm : 3 ≤ m + 3) :
1516   hosoya (prism (m+3)) =
1517     hosoya (prismDelSpoke (m+3) hm) + hosoya (pathGraph (m+2) □
    pathGraph 2) := by
1518   have h := hosoya_prism_split_spoke (m+3) hm
1519   have h2 : hosoya (SimpleGraph.induce
    (Set.univ \ {((⟨0, by omega⟩, (0 : Fin 2))), ((⟨0, by omega⟩, (1 :
    Fin 2))))) :
1520     Set (Fin (m+3) × Fin 2)) (prism (m+3))) =
1521     hosoya (pathGraph (m+2) □ pathGraph 2) :=
1522     hosoya_iso _ _ (prism_delSpokeEnds_induce_iso_ladder (m+3) hm)
1523   rw [h2] at h
1524   exact h
1525
1526
1527 -- ===== T4f 初值: a(4..6) = 71,228,733 与 c(3..6) = 4,15,46,150 =====
1528
1529 theorem a4 : hosoya (pathGraph 4 □ pathGraph 2) = 71 := by
1530   -- rec3 (m=1): a(4) + a(1) = 3·a(3) + a(2)
1531   have hr := hosoya_ladder_rec3 1
1532   have h4 : hosoya (pathGraph (1 + 3) □ pathGraph 2) = hosoya (pathGraph
    4 □ pathGraph 2) := rfl
1533   have h3 : hosoya (pathGraph (1 + 2) □ pathGraph 2) = hosoya (pathGraph
    3 □ pathGraph 2) := rfl
1534   have h2 : hosoya (pathGraph (1 + 1) □ pathGraph 2) = hosoya (pathGraph
    2 □ pathGraph 2) := rfl
1535   rw [h4, h3, h2] at hr
1536   have e1 := hosoya_lad1
1537   have e2 := hosoya_lad2
1538   have e3 := hosoya_lad3
1539   omega
1540
1541 theorem a5 : hosoya (pathGraph 5 □ pathGraph 2) = 228 := by
1542   -- rec3 (m=2): a(5) + a(2) = 3·a(4) + a(3)
1543   have hr := hosoya_ladder_rec3 2
1544   have h5 : hosoya (pathGraph (2 + 3) □ pathGraph 2) = hosoya (pathGraph
    5 □ pathGraph 2) := rfl
1545   have h4 : hosoya (pathGraph (2 + 2) □ pathGraph 2) = hosoya (pathGraph

```

```

1546   4  $\square$  pathGraph 2) := rfl
1547   have h3 : hosoya (pathGraph (2 + 1)  $\square$  pathGraph 2) = hosoya (pathGraph
1548     3  $\square$  pathGraph 2) := rfl
1549   rw [h5, h4, h3] at hr
1550   have e2 := hosoya_lad2
1551   have e3 := hosoya_lad3
1552   have e4 := a4
1553   omega
1554   theorem a6 : hosoya (pathGraph 6  $\square$  pathGraph 2) = 733 := by
1555     -- rec3 (m=3): a(6) + a(3) = 3·a(5) + a(4)
1556     have hr := hosoya_ladder_rec3 3
1557     have h6 : hosoya (pathGraph (3 + 3)  $\square$  pathGraph 2) = hosoya (pathGraph
1558       6  $\square$  pathGraph 2) := rfl
1559     have h5 : hosoya (pathGraph (3 + 2)  $\square$  pathGraph 2) = hosoya (pathGraph
1560       5  $\square$  pathGraph 2) := rfl
1561     have h4 : hosoya (pathGraph (3 + 1)  $\square$  pathGraph 2) = hosoya (pathGraph
1562       4  $\square$  pathGraph 2) := rfl
1563     rw [h6, h5, h4] at hr
1564     have e3 := hosoya_lad3
1565     have e4 := a4
1566     have e5 := a5
1567     omega
1568   theorem c3 : matchCount (PrismR2.Cst 3 3) = 4 := by decide
1569   theorem c4 : matchCount (PrismR2.Cst 4 4) = 15 := by decide
1570   theorem c5 : matchCount (PrismR2.Cst 5 5) = 46 := by decide
1571   /-- c6 递推链所需的 4 个小实例 ( $m \leq 2$  或与  $c_4$  同规模的 decide)。 -/
1572   private theorem c6_small :
1573     matchCount (PrismR2.V3 6 2) = 14  $\wedge$ 
1574     matchCount (PrismR2.V5 6 2) = 10  $\wedge$ 
1575     matchCount (PrismR2.V7 6 2) = 8  $\wedge$ 
1576     matchCount (PrismR2.Cst 6 (3 + 1)) = 15 := by
1577   refine {?_, ?_, ?_, ?_} <;>
1578     (try decide) <;>
1579     (try decide) <;>
1580     (try decide) <;>
1581     (try decide)
1582   /-- c(6) = 150: 直接 decide 超界 ( $2^{21}$  幂集枚举, whnf 心跳超时留档),
1583   改走 PrismHosoya 库件  $T1 - T7$  递推实例链 + omega 线性消元。 -/
1584   theorem c6 : matchCount (PrismR2.Cst 6 6) = 150 := by
1585     obtain {e32, e42, e72, c6c4} := c6_small
1586     set i3 : Fin 6 := ⟨3, by omega⟩ with hi3
1587     set i4 : Fin 6 := ⟨4, by omega⟩ with hi4
1588     --  $V5r(3) = V3(2) + V7(2) = 14 + 8 = 22$  ( $T1$   $m=2$ )
1589

```

```

1590 have s3 : matchCount (PrismR2.V5r 6 3 i3) = matchCount (PrismR2.V5r 6
      (2 + 1) i3) := rfl
1591 have A1 := PrismR2.T1 (N := 6) (m := 2) i3 (by simp [hi3]) (by omega) (
      by omega)
1592 rw [← s3] at A1
1593 have hV5r3 : matchCount (PrismR2.V5r 6 3 i3) = 22 := by omega
1594 --  $V5(3) = V5r(3) + V5(2) = 22 + 10 = 32$  (T7 m=2)
1595 have s5a : matchCount (PrismR2.V5 6 3) = matchCount (PrismR2.V5 6 (2 +
      1)) := rfl
1596 have s5b : matchCount (PrismR2.V5r 6 3 i3) = matchCount (PrismR2.V5r 6
      (2 + 1) i3) := rfl
1597 have A2 := PrismR2.T7 (N := 6) (m := 2) i3 (by simp [hi3]) (by omega) (
      by omega)
1598 rw [← s5a, ← s5b] at A2
1599 have hV53 : matchCount (PrismR2.V5 6 3) = 32 := by omega
1600 --  $V3r(3) = V5r(3) + Cst(4) = 22 + 15 = 37$  (T4 m=3)
1601 have A3 := PrismR2.T4 (N := 6) (m := 3) i3 (by simp [hi3]) (by omega) (
      by omega)
1602 have hV3r3 : matchCount (PrismR2.V3r 6 3 i3) = 37 := by omega
1603 --  $V3(3) = V3r(3) + V5(2) = 37 + 10 = 47$  (T3 m=3)
1604 have s7 : matchCount (PrismR2.V5 6 2) = matchCount (PrismR2.V5 6 (3 -
      1)) := rfl
1605 have A4 := PrismR2.T3 (N := 6) (m := 3) i3 (by simp [hi3]) (by omega) (
      by omega)
1606 rw [← s7] at A4
1607 have hV33 : matchCount (PrismR2.V3 6 3) = 47 := by omega
1608 --  $V7(3) = V3(2) + V5(2) = 14 + 10 = 24$  (T6 m=3)
1609 have s8a : matchCount (PrismR2.V3 6 2) = matchCount (PrismR2.V3 6 (3 -
      1)) := rfl
1610 have A5 := PrismR2.T6 (N := 6) (m := 3) (by omega) (by omega)
1611 rw [← s8a, ← s7] at A5
1612 have hV73 : matchCount (PrismR2.V7 6 3) = 24 := by omega
1613 --  $V5r(4) = V3(3) + V7(3) = 47 + 24 = 71$  (T1 m=3)
1614 have s9 : matchCount (PrismR2.V5r 6 4 i4) = matchCount (PrismR2.V5r 6
      (3 + 1) i4) := rfl
1615 have A6 := PrismR2.T1 (N := 6) (m := 3) i4 (by simp [hi4]) (by omega) (
      by omega)
1616 rw [← s9] at A6
1617 have hV5r4 : matchCount (PrismR2.V5r 6 4 i4) = 71 := by omega
1618 --  $V5(4) = V5r(4) + V5(3) = 71 + 32 = 103$  (T5 m=4)
1619 have s10 : matchCount (PrismR2.V5 6 3) = matchCount (PrismR2.V5 6 (4 -
      1)) := rfl
1620 have A7 := PrismR2.T5 (N := 6) (m := 4) i4 (by simp [hi4]) (by omega) (
      by omega)
1621 rw [← s10] at A7
1622 have hV54 : matchCount (PrismR2.V5 6 4) = 103 := by omega
1623 --  $Cst(6) = V5(4) + V3(3) = 103 + 47 = 150$  (T2 m=4)
1624 have s11a : matchCount (PrismR2.Cst 6 6) = matchCount (PrismR2.Cst 6 (4
      + 2)) := rfl

```

```

1625   have s11b : matchCount (PrismR2.V3 6 3) = matchCount (PrismR2.V3 6 (4 -
      1)) := rfl
1626   have A8 := PrismR2.T2 (N := 6) (m := 4) (by omega) (by omega)
1627   rw [← s11a, ← s11b] at A8
1628   omega
1629
1630 -- ===== T4f 强归纳 + R4 代数 (纯  $\mathbb{Z}$ , 探针逐字) =====
1631
1632   theorem aspoke_strong (D :  $\mathbb{N} \rightarrow \mathbb{Z}$ ) (h0 : D 0 = 25) (h1 : D 1 = 86) (h2 :
      D 2 = 271) (h3 : D 3 = 876)
1633   (hr :  $\forall k : \mathbb{N}, D (k+4) + D k = 2 * D (k+3) + 4 * D (k+2)$ ) :  $\forall m : \mathbb{N},$ 
      D m = aspoke m := by
1634     intro m
1635     induction m using Nat.strongRecOn with
1636     | ind n ih =>
1637       rcases Nat.lt_or_ge n 4 with hlt | hge
1638       · obtain rfl | rfl | rfl | rfl :=
1639         (show n = 0  $\vee$  n = 1  $\vee$  n = 2  $\vee$  n = 3 by omega)
1640       · exact h0
1641       · exact h1
1642       · exact h2
1643       · exact h3
1644       · obtain ⟨k, rfl⟩ :  $\exists k, n = k + 4 := \langle n - 4, \text{by omega} \rangle$ 
1645         have hrk := hr k
1646         have e3 := ih (k + 3) (by omega)
1647         have e2 := ih (k + 2) (by omega)
1648         have e0 := ih k (by omega)
1649         simp only [aspoke]
1650         omega
1651
1652   theorem arim_strong (D :  $\mathbb{N} \rightarrow \mathbb{Z}$ ) (h0 : D 0 = 26) (h1 : D 1 = 86) (h2 : D
      2 = 274) (h3 : D 3 = 883)
1653   (hr :  $\forall k : \mathbb{N}, D (k+4) + D k = 2 * D (k+3) + 4 * D (k+2)$ ) :  $\forall m : \mathbb{N},$ 
      D m = arim m := by
1654     intro m
1655     induction m using Nat.strongRecOn with
1656     | ind n ih =>
1657       rcases Nat.lt_or_ge n 4 with hlt | hge
1658       · obtain rfl | rfl | rfl | rfl :=
1659         (show n = 0  $\vee$  n = 1  $\vee$  n = 2  $\vee$  n = 3 by omega)
1660       · exact h0
1661       · exact h1
1662       · exact h2
1663       · exact h3
1664       · obtain ⟨k, rfl⟩ :  $\exists k, n = k + 4 := \langle n - 4, \text{by omega} \rangle$ 
1665         have hrk := hr k
1666         have e3 := ih (k + 3) (by omega)
1667         have e2 := ih (k + 2) (by omega)
1668         have e0 := ih k (by omega)

```

```

1669     simp only [arim]
1670     omega
1671
1672 theorem dspoke_r4 (a c :  $\mathbb{N} \rightarrow \mathbb{Z}$ )
1673   (ha :  $\forall k : \mathbb{N}, a (k+4) + a k = 2 * a (k+3) + 4 * a (k+2)$ )
1674   (hc :  $\forall k : \mathbb{N}, 3 \leq k \rightarrow c (k+4) + c k = 2 * c (k+3) + 4 * c (k+2)$ ) :
1675    $\forall k : \mathbb{N}, (a (k+7) + 2 * c (k+7) + a (k+5) - a (k+6)) + (a (k+3) + 2 * c (k+3) + a (k+1) - a (k+2))$ 
1676     =  $2 * (a (k+6) + 2 * c (k+6) + a (k+4) - a (k+5)) + 4 * (a (k+5) + 2 * c (k+5) + a (k+3) - a (k+4))$  := by
1677   intro k
1678   -- omega 原子坑规避: ha (k+j) 展开为 a (k+j+4) 等嵌套形, 须复述为与目标同形的 a (k+7) 等。
1679   have ha1 : a (k + 5) + a (k + 1) = 2 * a (k + 4) + 4 * a (k + 3) := by
1680     have h := ha (k + 1)
1681     rw [show k + 1 + 4 = k + 5 from by omega,
1682         show k + 1 + 3 = k + 4 from by omega,
1683         show k + 1 + 2 = k + 3 from by omega] at h
1684     exact h
1685   have ha2 : a (k + 6) + a (k + 2) = 2 * a (k + 5) + 4 * a (k + 4) := by
1686     have h := ha (k + 2)
1687     rw [show k + 2 + 4 = k + 6 from by omega,
1688         show k + 2 + 3 = k + 5 from by omega,
1689         show k + 2 + 2 = k + 4 from by omega] at h
1690     exact h
1691   have ha3 : a (k + 7) + a (k + 3) = 2 * a (k + 6) + 4 * a (k + 5) := by
1692     have h := ha (k + 3)
1693     rw [show k + 3 + 4 = k + 7 from by omega,
1694         show k + 3 + 3 = k + 6 from by omega,
1695         show k + 3 + 2 = k + 5 from by omega] at h
1696     exact h
1697   have hc3 : c (k + 7) + c (k + 3) = 2 * c (k + 6) + 4 * c (k + 5) := by
1698     have h := hc (k + 3) (by omega)
1699     rw [show k + 3 + 4 = k + 7 from by omega,
1700         show k + 3 + 3 = k + 6 from by omega,
1701         show k + 3 + 2 = k + 5 from by omega] at h
1702     exact h
1703   omega
1704
1705 theorem drim_r4 (a c :  $\mathbb{N} \rightarrow \mathbb{Z}$ )
1706   (ha :  $\forall k : \mathbb{N}, a (k+4) + a k = 2 * a (k+3) + 4 * a (k+2)$ )
1707   (hc :  $\forall k : \mathbb{N}, 3 \leq k \rightarrow c (k+4) + c k = 2 * c (k+3) + 4 * c (k+2)$ ) :
1708    $\forall k : \mathbb{N}, (a (k+7) + c (k+7)) + (a (k+3) + c (k+3)) = 2 * (a (k+6) + c (k+6)) + 4 * (a (k+5) + c (k+5))$  := by
1709   intro k
1710   have ha3 : a (k + 7) + a (k + 3) = 2 * a (k + 6) + 4 * a (k + 5) := by
1711     have h := ha (k + 3)
1712     rw [show k + 3 + 4 = k + 7 from by omega,
1713         show k + 3 + 3 = k + 6 from by omega,
```

```

1714         show k + 3 + 2 = k + 5 from by omega] at h
1715     exact h
1716     have hc3 : c (k + 7) + c (k + 3) = 2 * c (k + 6) + 4 * c (k + 5) := by
1717         have h := hc (k + 3) (by omega)
1718         rw [show k + 3 + 4 = k + 7 from by omega,
1719             show k + 3 + 3 = k + 6 from by omega,
1720             show k + 3 + 2 = k + 5 from by omega] at h
1721     exact h
1722     omega
1723
1724 -- ===== 主桥收口：具体序列实例化 + D_spoke/D_rim 强归纳 =====
1725
1726 /-- 具体序列 ( $\mathbb{Z}$  化) :  $a(n)$  = 梯子匹配数,  $c(n)$  = Cst 匹配数。 -/
1727 def az :  $\mathbb{N} \rightarrow \mathbb{Z}$  := fun k => ((hosoya (pathGraph k  $\square$  pathGraph 2) :  $\mathbb{N}$ ) :  $\mathbb{Z}$ )
1728 def cz :  $\mathbb{N} \rightarrow \mathbb{Z}$  := fun k => ((matchCount (PrismR2.Cst k k) :  $\mathbb{N}$ ) :  $\mathbb{Z}$ )
1729
1730 @[simp] theorem az_val (k :  $\mathbb{N}$ ) : az k = ((hosoya (pathGraph k  $\square$ 
1731     pathGraph 2) :  $\mathbb{N}$ ) :  $\mathbb{Z}$ ) := rfl
1732
1733 @[simp] theorem cz_val (k :  $\mathbb{N}$ ) : cz k = ((matchCount (PrismR2.Cst k k) :
1734      $\mathbb{N}$ ) :  $\mathbb{Z}$ ) := rfl
1735
1736 /-- a-R4 具体实例 (由 hosoya_ladder_rec4)。 -/
1737 theorem az_r4 (k :  $\mathbb{N}$ ) : az (k+4) + az k = 2 * az (k+3) + 4 * az (k+2) :=
1738     by
1739     have h := hosoya_ladder_rec4 k
1740     simp only [az_val]
1741     push_cast
1742     omega
1743
1744 /-- c-R4 具体实例 (由 c_rec4', ambient-free)。 -/
1745 theorem cz_r4 (k :  $\mathbb{N}$ ) (hk : 3  $\leq$  k) : cz (k+4) + cz k = 2 * cz (k+3) + 4
1746     * cz (k+2) := by
1747     have h := c_rec4' k hk
1748     simp only [cz_val]
1749     push_cast
1750     omega
1751
1752 /--  $D\_spoke(m) = a(m+3) + 2c(m+3) + a(m+1) - a(m+2)$  ( $d\_spoke$  的  $\mathbb{Z}$  表
1753     达)。 -/
1754 def Dspoke :  $\mathbb{N} \rightarrow \mathbb{Z}$  := fun m => az (m+3) + 2 * cz (m+3) + az (m+1) - az (
1755     m+2)
1756
1757 /--  $D\_rim(m) = a(m+3) + c(m+3)$  ( $d\_rim$  的  $\mathbb{Z}$  表达)。 -/
1758 def Drim :  $\mathbb{N} \rightarrow \mathbb{Z}$  := fun m => az (m+3) + cz (m+3)
1759
1760 @[simp] theorem Dspoke_val (m :  $\mathbb{N}$ ) :
1761     Dspoke m = az (m+3) + 2 * cz (m+3) + az (m+1) - az (m+2) := rfl
1762 @[simp] theorem Drim_val (m :  $\mathbb{N}$ ) : Drim m = az (m+3) + cz (m+3) := rfl
1763

```

```

1756 theorem Dspoke_r4 (k : ℕ) : Dspoke (k+4) + Dspoke k = 2 * Dspoke (k+3) +
    4 * Dspoke (k+2) := by
1757   simp only [Dspoke_val, Nat.add_assoc, Nat.reduceAdd] using
1758     dspoke_r4 az cz az_r4 (fun j hj => cz_r4 j hj) k
1759
1760 theorem Drim_r4 (k : ℕ) : Drim (k+4) + Drim k = 2 * Drim (k+3) + 4 * Drim
    (k+2) := by
1761   simp only [Drim_val, Nat.add_assoc, Nat.reduceAdd] using drim_r4 az cz
    az_r4 (fun j hj => cz_r4 j hj) k
1762
1763 -- 具体序列数值 (az1..az6 / cz3..cz6)
1764 theorem az1 : az 1 = 2 := by rw [az_val, hosoya_lad1]; norm_num
1765 theorem az2 : az 2 = 7 := by rw [az_val, hosoya_lad2]; norm_num
1766 theorem az3 : az 3 = 22 := by rw [az_val, hosoya_lad3]; norm_num
1767 theorem az4 : az 4 = 71 := by rw [az_val, a4]; norm_num
1768 theorem az5 : az 5 = 228 := by rw [az_val, a5]; norm_num
1769 theorem az6 : az 6 = 733 := by rw [az_val, a6]; norm_num
1770 theorem cz3 : cz 3 = 4 := by rw [cz_val, c3]; norm_num
1771 theorem cz4 : cz 4 = 15 := by rw [cz_val, c4]; norm_num
1772 theorem cz5 : cz 5 = 46 := by rw [cz_val, c5]; norm_num
1773 theorem cz6 : cz 6 = 150 := by rw [cz_val, c6]; norm_num
1774
1775 /-- D_spoke 初值: 25, 86, 271, 876。 -/
1776 theorem Dspoke0 : Dspoke 0 = 25 := by
1777   simp only [Dspoke_val, Nat.zero_add, az3, cz3, az1, az2]
1778   norm_num
1779 theorem Dspoke1 : Dspoke 1 = 86 := by
1780   simp only [Dspoke_val, Nat.reduceAdd, az4, cz4, az2, az3]
1781   norm_num
1782 theorem Dspoke2 : Dspoke 2 = 271 := by
1783   simp only [Dspoke_val, Nat.reduceAdd, az5, cz5, az3, az4]
1784   norm_num
1785 theorem Dspoke3 : Dspoke 3 = 876 := by
1786   simp only [Dspoke_val, Nat.reduceAdd, az6, cz6, az4, az5]
1787   norm_num
1788
1789 /-- D_rim 初值: 26, 86, 274, 883。 -/
1790 theorem Drim0 : Drim 0 = 26 := by
1791   simp only [Drim_val, Nat.zero_add, az3, cz3]
1792   norm_num
1793 theorem Drim1 : Drim 1 = 86 := by
1794   simp only [Drim_val, Nat.reduceAdd, az4, cz4]
1795   norm_num
1796 theorem Drim2 : Drim 2 = 274 := by
1797   simp only [Drim_val, Nat.reduceAdd, az5, cz5]
1798   norm_num
1799 theorem Drim3 : Drim 3 = 883 := by
1800   simp only [Drim_val, Nat.reduceAdd, az6, cz6]
1801   norm_num

```

```

1802
1803 /-- **主桥 (spoke 侧) **:  $\mathbb{Z}$  口径下  $hosoya (prismDelSpoke (m+3)) = D\_spoke(m)$ 。
1804 三式  $\omega$  消元:  $hosoya\_prism\_mc + delspoke\_eq + p\_decomp$ 。 -/
1805 theorem dspoke_cast (m :  $\mathbb{N}$ ) (hm :  $3 \leq m + 3$ ) :
1806   ((hosoya (prismDelSpoke (m+3)) hm) :  $\mathbb{N}$ ) :  $\mathbb{Z}$  = Dspoke m := by
1807     have h0 := hosoya_prism_mc m
1808     have h1 := delspoke_eq m hm
1809     have h2 := p_decomp m
1810     simp only [Dspoke_val, az_val, cz_val]
1811     push_cast
1812     omega
1813
1814 /-- **主桥 (rim 侧) **:  $\mathbb{Z}$  口径下  $hosoya (prismDelRim (m+3)) = D\_rim(m)$ 。
1815   -/
1816 theorem drim_cast (m :  $\mathbb{N}$ ) (hm :  $3 \leq m + 3$ ) :
1817   ((hosoya (prismDelRim (m+3)) hm) :  $\mathbb{N}$ ) :  $\mathbb{Z}$  = Drim m := by
1818     have h := delrim_eq m hm
1819     simp only [Drim_val, az_val, cz_val]
1820     push_cast
1821     omega
1822
1823 /-- 强归纳收口 (spoke 侧) :  $D\_spoke(m) = aspoke(m)$ 。 -/
1824 theorem dspoke_val (m :  $\mathbb{N}$ ) (hm :  $3 \leq m + 3$ ) :
1825   ((hosoya (prismDelSpoke (m+3)) hm) :  $\mathbb{N}$ ) :  $\mathbb{Z}$  = aspoke m := by
1826     have h0 := dspoke_cast m hm
1827     have h1 := aspoke_strong Dspoke Dspoke0 Dspoke1 Dspoke2 Dspoke3
1828       Dspoke_r4 m
1829     omega
1830
1831 /-- 强归纳收口 (rim 侧) :  $D\_rim(m) = arim(m)$ 。 -/
1832 theorem drim_val (m :  $\mathbb{N}$ ) (hm :  $3 \leq m + 3$ ) :
1833   ((hosoya (prismDelRim (m+3)) hm) :  $\mathbb{N}$ ) :  $\mathbb{Z}$  = arim m := by
1834     have h0 := drim_cast m hm
1835     have h1 := arim_strong Drim Drim0 Drim1 Drim2 Drim3 Drim_r4 m
1836     omega
1837
1838 end PrismR3
1839
1840 -- ===== 冻结定理头 (statement.lean L90-98 逐字, 占位符已替换为实证证明
1841   体) =====
1842
1843 /-- **主定理一 (spoke 侧桥定理) **: 对一切  $n \geq 3$ , 删一条 spoke 的棱柱的匹
1844   配数等于序列
1845   `aspoke (n - 3)`。其中 `aspoke` 为已冻结序列 (初值 25, 86, 271, 876, 四阶递
1846   推
1847   ` $a(n+4) = 2 \cdot a(n+3) + 4 \cdot a(n+2) - a(n)$ `, 下标 0 对应图侧  $n = 3$ )。 -/
1848 theorem hosoya_prism_del_spoke :  $\forall (n : \mathbb{N}) (hn : 3 \leq n)$ ,
1849   (hosoya (prismDelSpoke n hn) :  $\mathbb{Z}$ ) = aspoke (n - 3) := by

```

```

1845   intro n hn
1846   obtain ⟨m, rfl⟩ : ∃ m, n = m + 3 := ⟨n - 3, by omega⟩
1847   rw [show m + 3 - 3 = m from by omega]
1848   exact PrismR3.dspeek_val m hn
1849
1850 /-- **主定理二 (rim 侧桥定理) **: 对一切  $n \geq 3$ , 删一条 rim 边的棱柱的匹配
      数等于序列
1851  $\text{arim}(n - 3)$ 。其中  $\text{arim}$  为已冻结序列 (初值 26, 86, 274, 883, 同四阶递
      推)。 -/
1852 theorem hosoya_prism_del_rim : ∀ (n : ℕ) (hn : 3 ≤ n),
1853   (hosoya (prismDelRim n hn) : ℤ) = arim (n - 3) := by
1854   intro n hn
1855   obtain ⟨m, rfl⟩ : ∃ m, n = m + 3 := ⟨n - 3, by omega⟩
1856   rw [show m + 3 - 3 = m from by omega]
1857   exact PrismR3.drim_val m hn

```

4 Verification evidence

- Toolchain: Lean v4.34.0 (leanprover/lean4:v4.34.0), mathlib4 v4.34.0, revision 5ed29652. Reproduction: with that pin, a strict compile of `proofs/bridge.lean` exits with code 0 and no error; the remaining diagnostics are linter-style warnings only (unused tactic), which the pipeline's discipline does not treat as grounds for rejection.
- Statement integrity: the frozen statement file (124 lines) has sha256

d186ff1651a4383e7e213b42ba4ec8c01
71d9e7004231db2764370cd967ca39e,

and the final proof file (1857 lines) has sha256

399ee81fe363f639811a3408397b14259
3ea1e5d195f3e916fd7b7c07af6c1.

The two frozen theorem heads inside the proof file are byte-identical to lines 90–91 and 96–97 of the frozen statement file (programmatic comparison, recorded in the source task's final adjudication); the base definitions at lines 28–47 of the proof file match the frozen snapshot's lines 64–83 verbatim. The statement excerpt in Listing 1 is an ordered subsequence of the frozen file with the three declared exclusions and no other deviation (verified mechanically by the package-local copy of `inspect-tex.py` under `audit/`; record in `audit/listing-verify.txt`).

- Kernel check: strict compilation exits 0 with no `sorry` and no `admit` (whole-file scan including comments: zero hits). `#print axioms` output, verbatim (11 declarations, independent re-print of 2026-10-01; the narrower monospace is typesetting only):

```

1 'hosoya_prism_del_spoke' depends on axioms: [propext, Classical.
      choice, Quot.sound]
2 'hosoya_prism_del_rim' depends on axioms: [propext, Classical.
      choice, Quot.sound]
3 'PrismR3.delspoke_eq' depends on axioms: [propext, Classical.choice
      , Quot.sound]
4 'PrismR3.delrim_eq' depends on axioms: [propext, Classical.choice,
      Quot.sound]
5 'PrismR3.cell_C4' depends on axioms: [propext, Classical.choice,
      Quot.sound]
6 'PrismR3.c5' depends on axioms: [propext, Classical.choice, Quot.
      sound]
7 'PrismR3.c6' depends on axioms: [propext, Classical.choice, Quot.
      sound]
8 'PrismR3.dspoke_val' depends on axioms: [propext, Classical.choice,
      Quot.sound]
9 'PrismR3.drim_val' depends on axioms: [propext, Classical.choice,
      Quot.sound]
10 'PrismR3.aspoke_strong' depends on axioms: [propext, Quot.sound]
11 'PrismR3.arim_strong' depends on axioms: [propext, Quot.sound]

```

Every line is a subset of the whitelist `{propext, Quot.sound, Classical.choice}`; `sorryAx` does not occur, and no `Lean.ofReduceBool` occurs (the proof contains no `native_decide`; the four `native_decide` data anchors of the frozen statement file are verification-only anchors outside the deposited proof, exactly as the frozen file's own header notes).

- Grading by the audit department: both frozen theorems are *complete proof* (final adjudication of 2026-10-01, seven checks all passed: independent strict recompile, independent axiom audit over 11 declarations, freeze check, redeclaration/pollution check, non-degeneracy spot checks, library byte-identity, boundary check; the adjudication text ships as `audit/final-phase2-r3.txt` in this deposit). No weakening, no added hypotheses, no unproven residue.
- Audit chain (originals in this deposit under `audit/` unless noted): the freeze-time sha256 record and the byte-wise symmetric-difference conclusion are carried by the source task's audit trail in the working repository; this deposit carries the final adjudication copy, the ax-

iom re-print, the listing byte-exactness record, and this note’s claim check `claim-check.md` — added to `audit/` after the audit department returns its adjudication, and a prerequisite for publication.

5 Novelty evidence

Adjudication copied from the pipeline’s exclusivity review (C9 gate): verdict **no occupying record found (reachable-channel scope)**. At the sequence level the review covers 41 ledgered queries over six forms of the two slices plus alias keywords (screening dossier, 58 raw responses), and the graph-side increment adds 31 queries over six channels with five positive controls, all passing; the strongest new sequence-side evidence is the exhaustive sweep of the OEIS linear-recurrence index page for the signature $(2, 4, 0, -1)$: exactly four sequences are registered under that signature (A102080 the mother prism sequence, A020877 the Möbius-ladder matchings, and two tiling/array families), and neither slice is among them. At the graph-side proposition level (prism with one edge deleted, endpoints kept, matching count satisfying the fourth-order recurrence) the arXiv / Crossref / MSE / GitHub / local mathlib channels returned no same-form statement. Value tier: **new sequence** — the “new sequence” half holds (both slices unregistered), the “new recurrence” half does *not* hold and is not claimed: the recurrence itself is the mother sequence A102080’s registered formula (same signature, different initial values; the review’s same-spectrum-different-initial-values wording, copied verbatim). The graph-side semantic bridge is graded **first formalisation (Lean three-layer zero same-form)**. The full query log, including every zero-hit query, is in this deposit at `audit/c9-record.md` (verbatim, byte-for-byte, three source records concatenated with their headers and adjudication conclusions unchanged).

channel	query	result
OEIS: sequence (screening)	all forms of the two slice forms, drop-first variants, interleavings, subchannels	all zero hit: number A102080 confirmed in register [3]
OEIS: sequence (alias)	sequence (2, 4, 0, -1) with page	4 registered entries, another also among them; strongest sequence-side channel
OEIS: keyword (alias)	alias and keyword strings	zero hit: object hit
arXiv	graph-side 0 + screening layer	nearest neighbours read at distance (full-text level (perfect-matching structural properties, PB-property, keyword collation): no matching count, no edge deletion
Crossref	graph-side 1	one family-related record (Alford / Pinner), matching criterion in prime graphs [5]; structural <=>isomorphism, not a counting result; full text behind paywall, recorded as normal
Mathematics Stack Exchange	graph-side 4 + control	control hit, also queries zero
GitHub: Lean repository	8	zero same-form, prime, linear, matching-count intersections all empty; one prime-related Lean formalisation found and excluded on different form (domination number, not matching count)
local mathlib (pinned v4.34.0)	5	no prime definition, no linear word family, matching predicate only (no counting, APF)
mathlib repository	5	found full-text, linear (overridden) and general with search not reachable, recorded as <i>unrelated</i> , not as evidence: Alford / Pinner and the hidden-side Pinner 1983 record (object = linkers, not primes) remain normal

The channels are named so a later reader can re-run them; if any channel later returns a same-form record, the tier above weakens accordingly, and this deposit will be amended by a later version rather than by editing this record. A zero hit is not novelty; the strongest claim licensed here is that no record of this object was found in the channels queried.

6 Scope and limitations

Grade and boundary, stated as the audit states them. (a) The kernel layer proves Theorems 1 and 2 and nothing else beyond the supporting declarations contained in the proof file. (b) The grading of both is *complete proof* (0 **sorry**, axiom whitelist, no scaffold residue); this note makes no claim about recurrence minimality, closed forms, asymptotics, or the mod- k residue patterns of the sequences (the congruence layer of the parent program is not part of this deposit). (c) The identification of the counting function with the Hosoya index is definitional in the formalisation (**hosoya** is defined as that edge-subset count); no further combinatorial conjecture layer is carried. (d) The four **native_decide** data anchors of the frozen statement file ($n = 3, 4$ values 25, 86, 26, 86) are verification-only anchors and are not part of the deposited proof or its axiom set. (e) The novelty record’s literature layer excludes paywalled full texts; the recorded exposures (Aldred–Plummer 2017 full text; the ladder-side Farrell 1983 record, whose object is ladders not prisms) are recorded as unread rather than as evidence, and no universal claim of the form “the literature has not studied this” is made. (f) This note contains no literature review; it is a deposit record, not a survey paper.

Data availability

Lean sources, verification and novelty-search evidence are deposited on Zenodo; the DOI is reserved at upload by the author and is recorded in `metadata/README.md` of this package once created; see `UPLOAD.md` in the source repository. The deposit contains the claim note (LaTeX source and PDF), the Lean sources (**proofs/**: frozen statement file, final proof file, **lean-toolchain**), and the audit and novelty-search records (**audit/**). Claim note under CC BY 4.0, code under MIT.

Use of generative AI

- *AI-performed stages*: candidate generation and the exclusivity review, statement formalisation and freezing, the semantic roundtrip check, three-phase proof search (worker agents plus main-agent scaffold assembly under the pipeline’s verified-scaffold clause, provenance recorded in the proof file header and the task ledger), and the assembly of this note including its verbatim listings.

- *Model, node, budget*: the proving and audit phases ran on the session node (wire designation `glm-5.3` as recorded in the task ledger’s node-drift incident line), with no undeclared cross-node switching; consumed: agent-run samples 12 of the task-level permitted 20 (Phase 0: 2, Phase 1: 6, Phase 2: 12 – of which the two bridge theorems are the Phase 2 r3 close-out), formal kernel compiles 17 of the permitted 36, llm-calls 4 of 8 (plus 4 transmission failures, ledgered separately), and compile-debug probes 23 (single-entry category, no cap); LaTeX 0 on the source task. The numbers are copied from the source task’s `budget.log` (tally line of 2026-10-01, categories as ledgered there). This note consumed 0 pipeline LLM calls against a cap of 4; drafting was mechanical assembly.
- *Final adjudication*: all statements and proofs reported here were checked by the Lean 4 kernel: every proof compiles with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. Statement text was frozen before proving and compared byte-wise afterwards.
- The author reviewed the material and is responsible for all content. AI systems are not listed as authors.

Author contributions

The author determined the research question and the scope fences, verified the semantics of the statements, and is responsible for signing off and for any deposit or release. The automated pipeline (LLM-driven) generated the candidate propositions, the formalisation, the proof search and the draft text. No external release is performed by the pipeline.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *Automated Deduction – CADE-28*, LNCS, pp. 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37
- [2] The mathlib Community. The Lean mathematical library. *CPP 2020*, pp. 367–381, 2020. <https://doi.org/10.1145/3372885.3373824>
- [3] OEIS Foundation. Number of matchings in the $C_n \times P_2$ (n -prism) graph (sequence A102080; the mother sequence, registered under the same

fourth-order recurrence with different initial values). *The On-Line Encyclopedia of Integer Sequences*, entry A102080. <https://oeis.org/A102080>

- [4] R. E. L. Aldred and M. D. Plummer. Matching extension in prism graphs. *Discrete Applied Mathematics* 221 (2017), pp. 25–32. <https://doi.org/10.1016/j.dam.2016.12.017>